

21CSE524T COMPUTER VISION TECHNIQUES

School of Computing
College of Engineering and Technology
SRM Institute of Science and Technology, Kattankulathur

Course Outcomes (CO)

- **At the end of this course, learners will be able to:**
 - understand computer vision including fundamentals of image formation
 - learn about image formation
 - apply image processing techniques
 - calibrate computational photography
 - learn about Image rendering

Unit-5 – Image Rendering

- Motion models
 - Planar perspective motion
 - Rotational panoramas
 - Gap closing
 - Cylindrical and spherical coordinates
- Global alignment
 - Bundle adjustment
 - Parallax removal
 - Recognizing panoramas
- Compositing
- T9: Case Study
- T10: Case Study

Image-Based Rendering

- **Image-Based Rendering (IBR)** is a set of techniques in computer graphics and computer vision that **generate new views of a scene from a set of photographs or images**, rather than relying solely on full 3D models.
- It lies between traditional image processing and full 3D rendering, enabling realistic visualizations when 3D geometry is incomplete or expensive to capture.

Image-based Rendering

- In image-based rendering, 3D reconstruction techniques from computer vision are combined with computer graphics rendering techniques that use multiple views of a scene **to create interactive photo-realistic experiences**, such as the Photo Tourism system

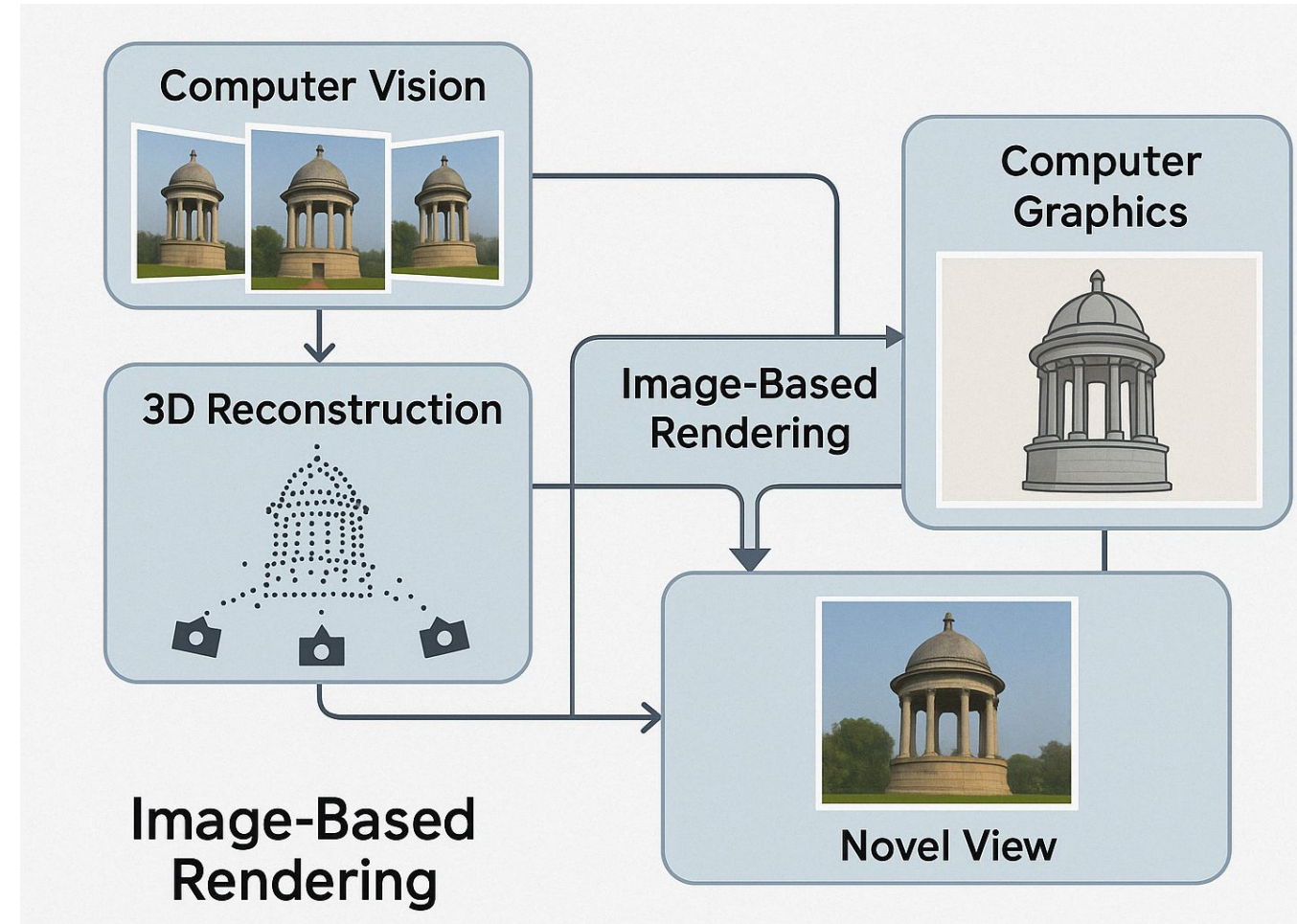


Image-Based Rendering- Example scenario

Goal: Create a **new view** of the Eiffel tower that wasn't captured in the original photos.

- **Input:**
 - The 3D geometry and camera poses from SfM.
 - The original images.
- **Process:**
 - Choose a virtual camera position.
 - Warp or reproject pixels from nearby images to the new viewpoint using the geometry.
 - Blend textures and fill gaps.
- **Output:** A **rendered photo-realistic image** as if taken from a new angle.



How IBR Works:

At its core, IBR follows these steps (method):

1. Capture Multiple Images

1. Acquire images from different viewpoints (e.g., a camera moving around an object or scene).

2. Estimate or Use Depth

Use stereo vision, LiDAR, structure-from-motion (SfM), or depth sensors to estimate the **depth** at each pixel of an image.

How IBR Works:

3. Reproject or Warp Images

1. Using **camera parameters** (intrinsics and extrinsics) and depth, project pixels from one image to a new viewpoint.
2. This is often called **image warping**.

4. Blend or Composite Views

1. Combine information from multiple images to fill in occlusions or gaps (using **blending, interpolation, or machine learning**).

Image Stitching

- A foundational technique in computer vision used for creating high-resolution photo mosaics and panoramas.
- **What is Image Stitching?**
 - **Image stitching** is the process of aligning and blending multiple overlapping images to produce a seamless, high-resolution composite image — often used in panoramic photography, digital maps, and satellite imagery.



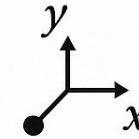
Components of Modern Image Stitching

- Image Alignment
- **Motion Models**
- Feature Detection & Matching
- **Bundle Adjustment**
- Image Warping & Compositing Surface
- Blending and Seam Removal
- Parallax and Ghost Removal



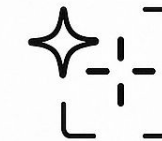
Image Alignment

Determines geometric relationship between images



Motion Models

Defines transformation of pixels across images



Feature Detection & Matching

Finds correspondences between keypoints

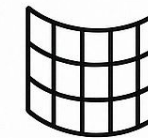


Image Warping & Compositing Surface

Warpes images to cylindrical or spherical surface



Blending and Seam Removal

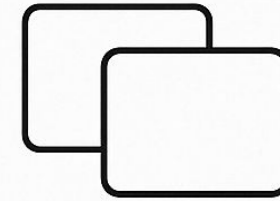
Corrects misalignment due to motion

Motion models

- Motion models define how pixels from one image correspond to pixels in another image, especially when there's camera movement or object motion between captures.
- They are mathematical transformations that help align multiple images into a single seamless view.

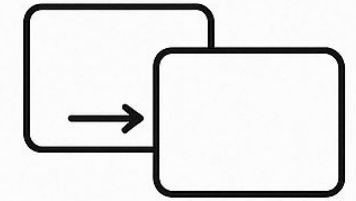
- **Types of Motion Models**

- 2D Parametric Transforms
- Homography (Planar Perspective)
- Pure Rotation Homography
- Cylindrical and Spherical Projections



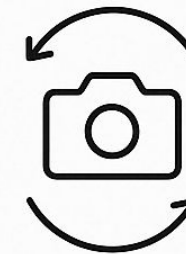
2D Affine

2D transform
(translation, rotation,
shear)



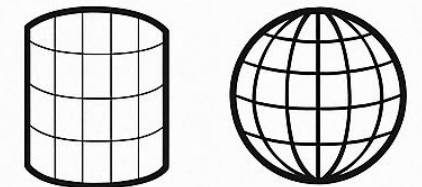
Homography

Planar perspective
transform



Pure Rotation

3D rotation around
optical center



Cylindrical and Spherical Projection

Maps to curved
surfaces

Motion models: 2D Parametric Transforms

Definition:

- A 2D parametric transform can be expressed as:

$$p' = T(p) \text{ where}$$

(p) is the original point in the 2D space, and

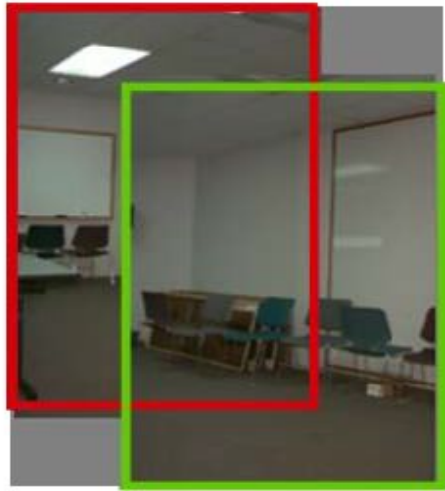
(p') is the transformed point after applying the transformation (T).

Types of Transformations

- **Affine Transformations:** These preserve points, straight lines, and planes.
- Common types include translation, scaling, rotation, and shearing.
- They can be represented using a 2×2 matrix.
- **Homographic Transformations:** These preserve lines but not necessarily parallelism.
- They can be used for perspective corrections and image stitching.

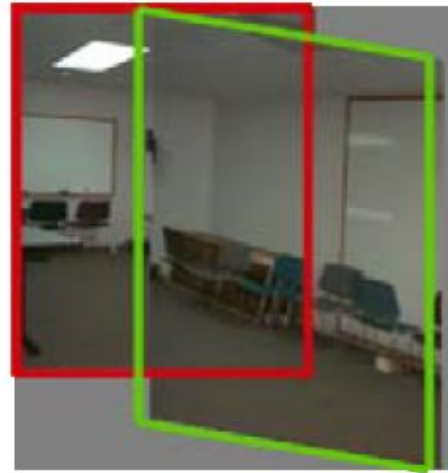
Motion models

- 2D Parametric Transforms
- Two-dimensional motion models and how they can be used for image stitching



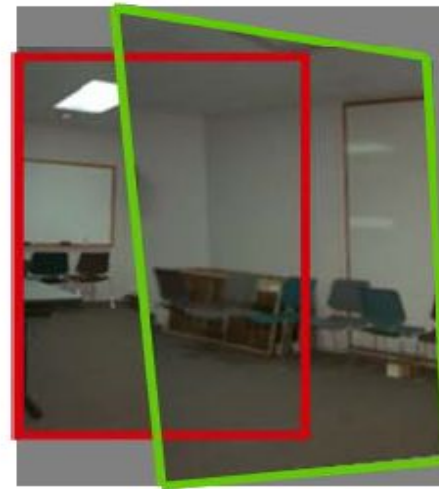
(a) translation [2 dof]

Shift image in X and Y. No scaling, rotation, or skew.



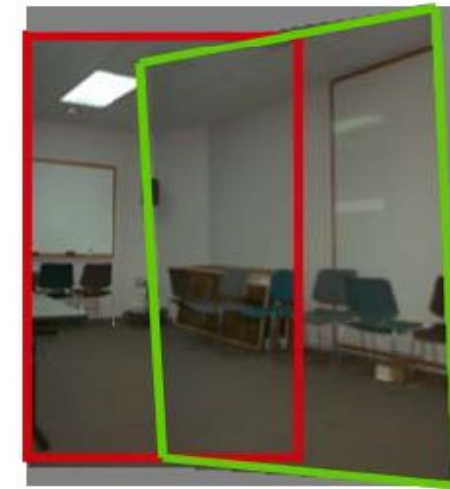
(b) affine [6 dof]

Allows rotation, translation, scaling, and shearing. Preserves parallel lines but not angles. 2 for translation, 1 for rotation, 1 for scaling in x, 1 for scaling in y, and 1 for shear



(c) perspective [8 dof]

8 independent values in a 3x3 matrix (scale-invariant)

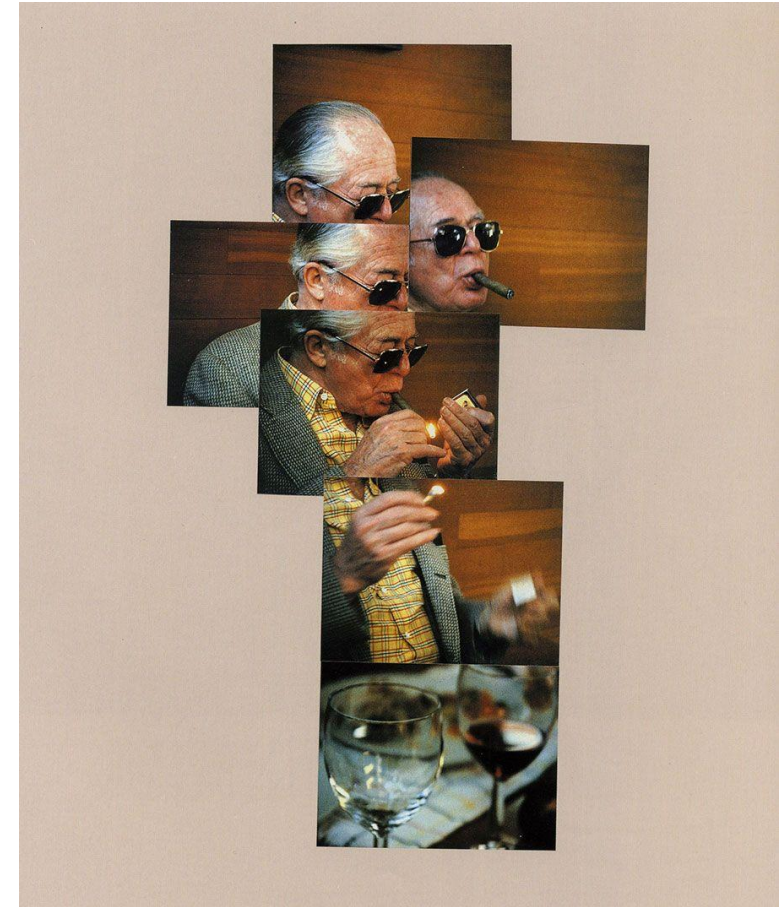


(d) 3D rotation [3+ dof]

Full 3D motion: 3 for rotation, 3 for translation. Used in SfM, SLAM, and pose estimation.

Planar perspective motion

- **Homography** - Planar perspective motion assumes that **all points in a scene lie on a flat (planar) surface**, and we want to map the transformation from one image to another when a camera moves or rotates.
- This is especially useful for:
 - **Image stitching** of flat scenes like walls, documents, aerial maps.
 - **Photo mosaics** with slight motion or rotations between frames.
 - **Panography** or artistic collages (like David Hockney's "joiners").
- It relies on **2D transforms**, including **translation**, **rotation**, and **scaling**, plus the effects of **perspective** from camera motion.



Planar perspective motion

- **Mathematical Model: Homography**

- The core mathematical model for planar perspective motion is a **homography matrix** — a 3×3 matrix that maps points from one image to another:

$$\tilde{x}_1 \approx H_{10}\tilde{x}_0$$

Where:

- $\tilde{x}_0$ is a point in the first image (in homogeneous coordinates).
 - $\tilde{x}_1$ is the corresponding point in the second image.
 - H_{10} is the **homography matrix** derived from the camera parameters and the plane equation.
- This mapping assumes that the scene is flat (planar) and the transformation includes projective effects (not just affine).

Planar perspective motion

- The following equations describe how each pixel in image 1 maps to image 2 when viewed under planar perspective motion:

Given the homography parameters h_{ij} , the transformation of pixel coordinates is:

$$x_1 = \frac{(1 + h_{00})x_0 + h_{01}y_0 + h_{02}}{h_{20}x_0 + h_{21}y_0 + 1}$$

$$y_1 = \frac{h_{10}x_0 + (1 + h_{11})y_0 + h_{12}}{h_{20}x_0 + h_{21}y_0 + 1}$$

$$H = \begin{bmatrix} 1 + h_{00} & h_{01} & h_{02} \\ h_{10} & 1 + h_{11} & h_{12} \\ h_{20} & h_{21} & 1 \end{bmatrix}$$

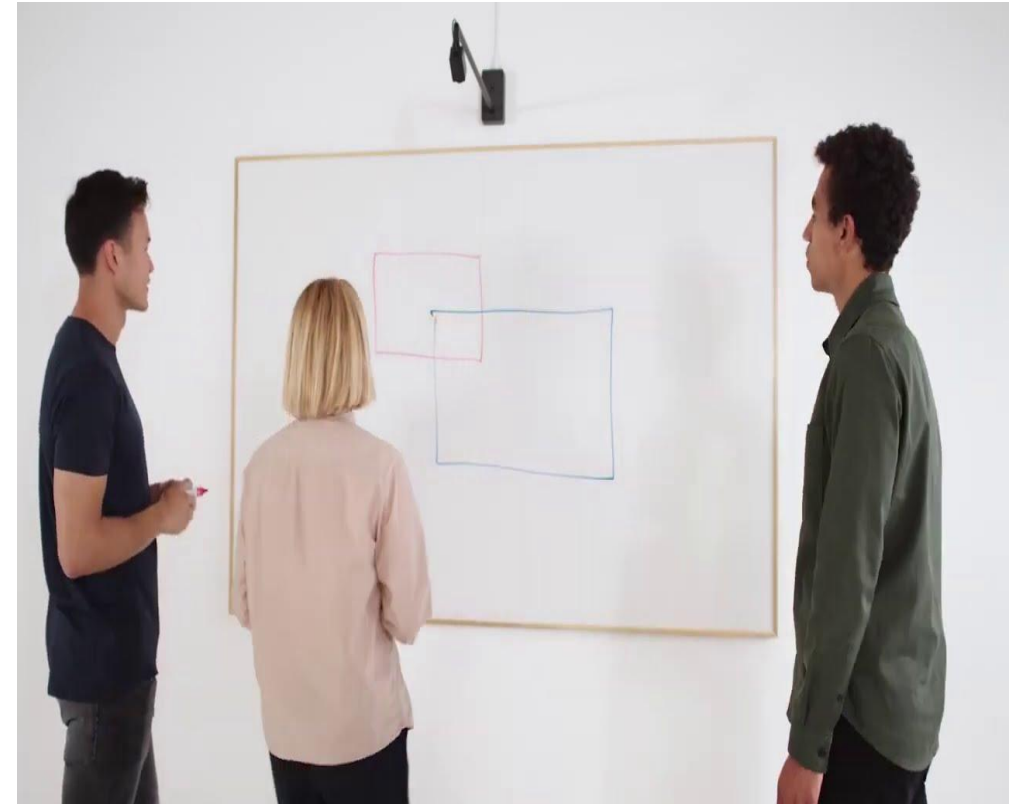
$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \sim H \cdot \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix}$$

Application: Whiteboard and Document Scanning

- When documents, maps, or artwork are too large to scan in one shot, we can use image stitching techniques to digitally combine multiple overlapping scans or photos into a single seamless composite.
- **Flatbed Scanner Example (Rigid Transform Model)**
 - Capture multiple scans with sufficient overlap.
 - Extract and match features between overlapping pairs (e.g., SIFT, ORB).
 - Estimate 2D rigid transformation between adjacent images:
$$x_{k+1} = R_k x_k + t_k$$
 Where, R_k : 2x2 rotation matrix and t_k is translation vector
 - Use RANSAC to handle noise and get a good inlier match set.
 - Warp all images to a common compositing surface (aligned with the first scan).
 - Blend overlapping regions using techniques like averaging or multi-band blending.

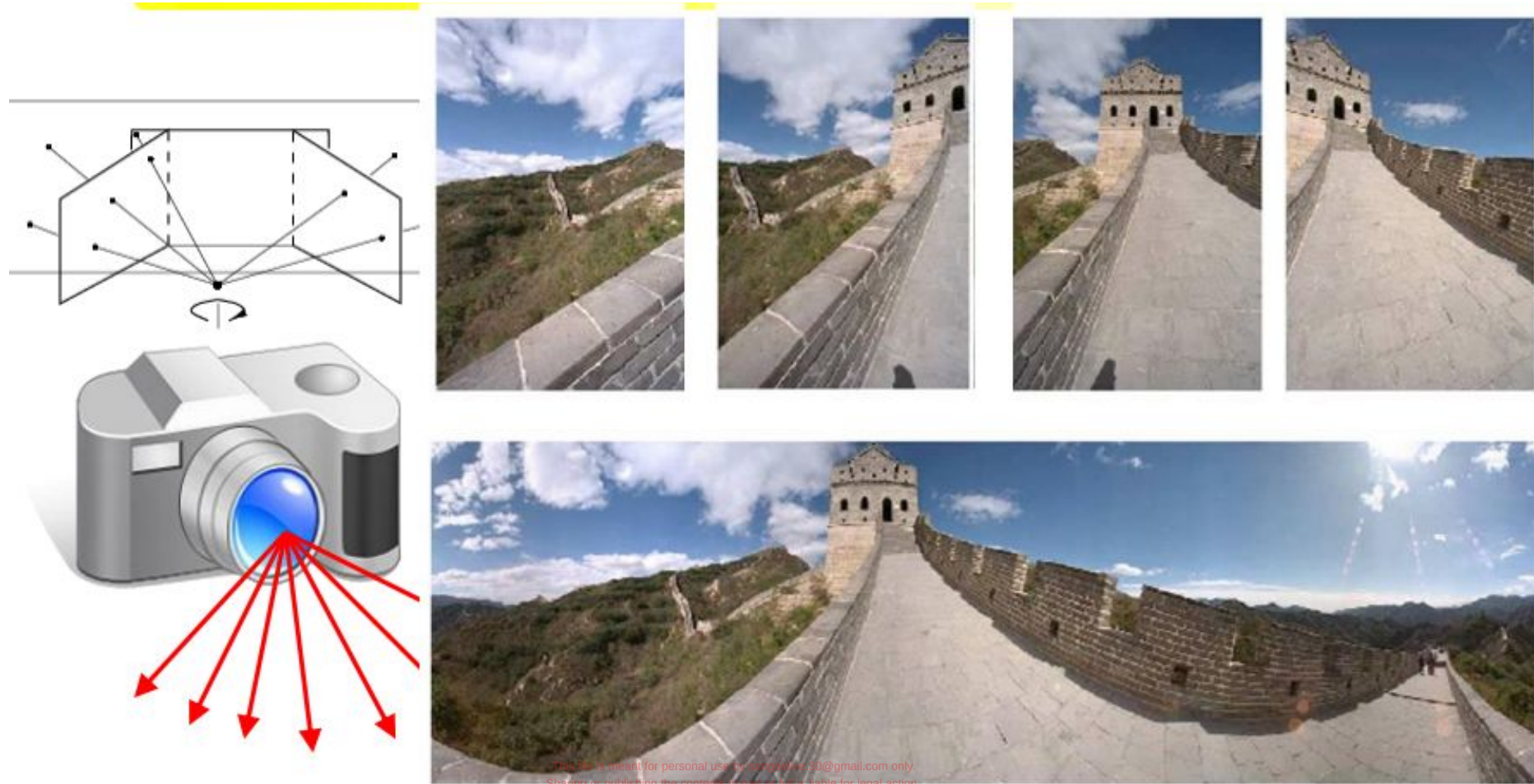
Application: Whiteboard and Document Scanning

- **Whiteboard Capture with Handheld Camera**
 - When photographing a whiteboard:
 - Surface is **planar**, but viewed from **different angles**.
 - Use a **homography** instead of a simple rigid transform.
- $x_1 \sim Hx_0$
- Optionally: Estimate **3D rigid motion + focal length** if camera parameters are unknown.



Panoramas

- Obtain a wider angle view by combining multiple images



How to stitch together a panorama?

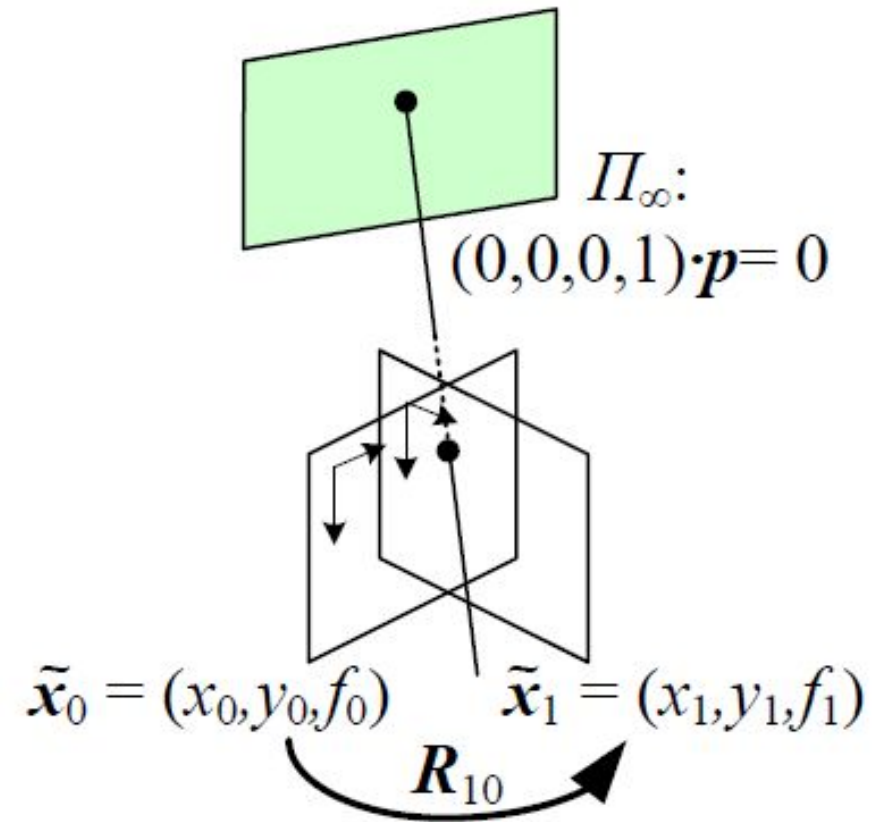
- Take a sequence of images from the same position
 - Rotate the camera about its optical center
- Compute transformation between second image and first
- Transform the second image to overlap with the first
- Blend the two together to create a mosaic
- If there are more images, repeat

Rotational panoramas



Rotational panoramas

- **Rotational panoramas** are created when a camera rotates **in place** — that is, around its **optical center** — while capturing a series of images.
- This setup is common in handheld panoramic shots, or when using tripods to sweep across a wide scene like the **Grand Canyon**.
- In this scenario, since the scene is distant, all points lie approximately on the **plane at infinity**, and we can **ignore translation** — only **rotation** matters.



Pure 3D camera rotation. The form of the homography (mapping) is particularly simple and depends only on the 3D rotation matrix and focal lengths.

Rotational panoramas

Mathematical Foundation

- When the camera undergoes **pure rotation**, the mapping between image pixels is governed by a simplified **homography**:

$$\tilde{H}_{10} = K_1 R_{10} K_0^{-1}$$

- Where:
 - $R_{10} = R_1 R_0^{-1}$ is the relative **3D rotation matrix** between views.
 - K is the **intrinsic matrix** of the camera (assumed to be diagonal and centered at the optical center)
- This reduces the number of parameters from 8 (as in general homography) to:
 - **3** if focal length is known
 - **4–5** if focal length is fixed or variable

Rotational panoramas

- Rotation Update (Incremental Refinement)
 - To refine image alignment, we **update the rotation matrix** incrementally using a **small rotation vector** $\omega = (\omega_x, \omega_y, \omega_z)$.
 - This is done using:

$$\tilde{H}(w) = D\tilde{H}_{10}$$

- Where

$$D = K_1 R(w) K_1^{-1}$$

- Using the small-angle approximation:

$$R(w) \approx I + [w]_x$$

- Which expands to:

$$D = \begin{bmatrix} 1 & -\omega_z & f_1\omega_y \\ \omega_z & 1 & -f_1\omega_x \\ -\omega_y/f_1 & \omega_x/f_1 & 1 \end{bmatrix}$$

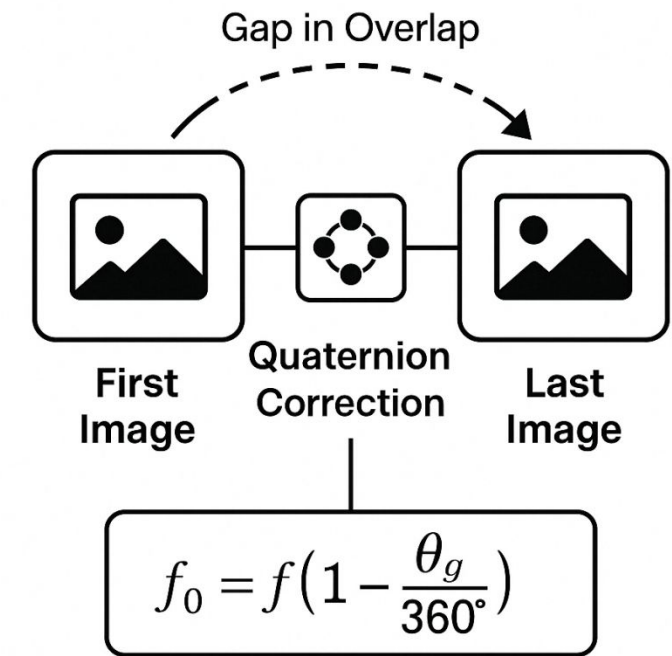
- This matrix allows us to:
 - **Estimate small corrections** to the current rotation
 - **Iteratively refine** the alignment

Gap closing

- When creating a **360° panoramic image** by rotating the camera and capturing overlapping frames, you might expect the **first and last images** to align perfectly.
- However, due to **accumulated errors** in rotation and focal length estimation, there's often a small **gap** or **overlap** — this is the **gap closing problem**.
- **What Causes the Gap?**
 - Errors in estimating focal length
 - **Drift** in incremental rotation estimation
 - Small misalignments between consecutive frames, which **accumulate**
 - Inaccuracies in camera calibration

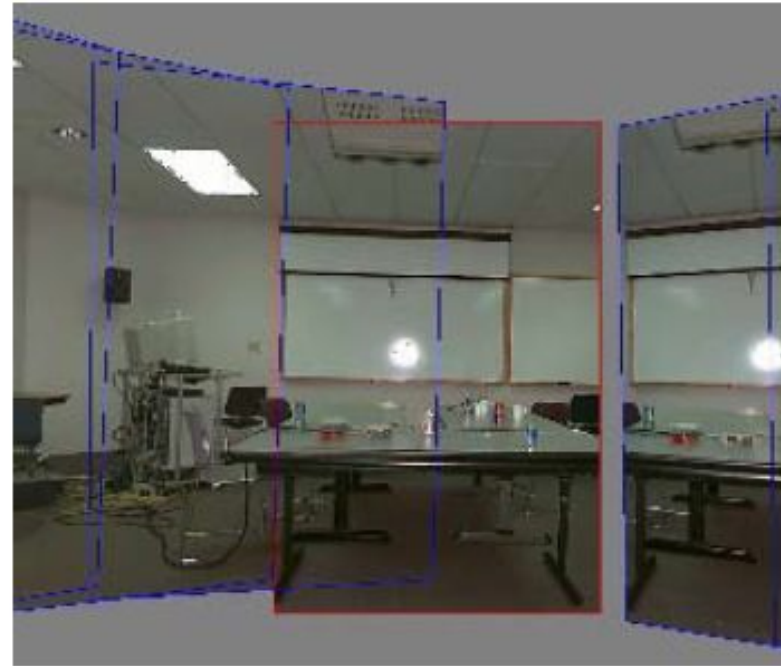
How Is Gap Closing Solved?

- **Compare First and Last Images**
 - Align the **last image** with the **first image** again.
 - Calculate the **rotation mismatch** between the two.
- **Use Quaternions for Smooth Correction**
 - Convert the two rotation matrices into **quaternions**.
 - Compute the “**error quaternion**” that represents the rotational discrepancy.
 - Divide this quaternion correction by the number of images in the loop.
 - **Distribute** the correction across all images to evenly fix the drift.
- **Update Focal Length (Optional)**
 - If the focal length was incorrectly estimated (e.g., **f = 510 instead of 468**), then:
 - Convert the **gap angle** θ_g into a focal length correction:
 - $$f_0 = f \left(1 - \frac{\theta_g}{360^\circ} \right)$$
 - This corrects focal length based on the **angular error** between the first and last image.

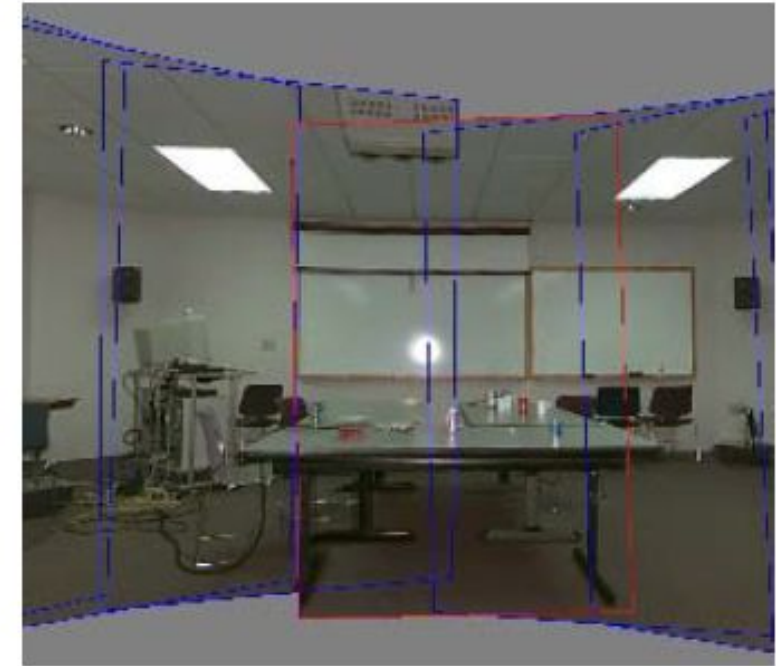


Example: Gap closing

Fig(a) Shows a gap between first and last images due to focal length error (9% off).



(a)



(b)

In Fig(b), After applying **gap closing**, the panorama is smooth and continuous.

CYLINDRICAL AND SPHERICAL COORDINATES

Cylindrical coordinates

- An alternative to using homographies or 3D motions to align images is to first warp the images into **cylindrical coordinates** and then use a pure translational model to align them

- **Mapping from image to cylinder:**

- Given a pixel ray (x,y,f) , the cylindrical projection is:

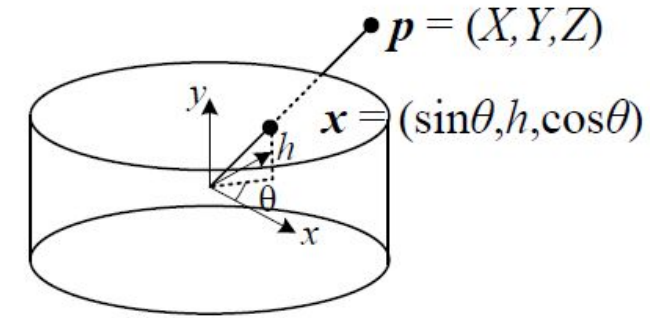
$$\begin{aligned} x' &= s\theta = s \tan^{-1} \frac{x}{f}, \\ y' &= sh = s \frac{y}{\sqrt{x^2 + f^2}}, \end{aligned}$$

Where:

s : radius of the cylinder (often set to focal length f)

x,y : image coordinates

f : focal length



- This mapping "unwraps" the image onto a cylindrical surface.
- Alignment becomes easier — often just **horizontal translation** between warped images.

- **Inverse mapping (cylinder → image):**

$$x = f \cdot \tan \left(\frac{x'}{s} \right)$$

$$y = \frac{y'}{s} \cdot f \cdot \sec \left(\frac{x'}{s} \right)$$

Spherical Coordinates

- Images can also be projected onto a spherical surface, which is useful if the final panorama includes a full sphere or hemisphere of views, instead of just a cylindrical strip.
- Used for **full 360° panoramas**, covering **both horizontal and vertical field of view** (like turning your head in all directions).
- Given a pixel ray (x,y,f) , map to a sphere via angles (θ,ϕ) :

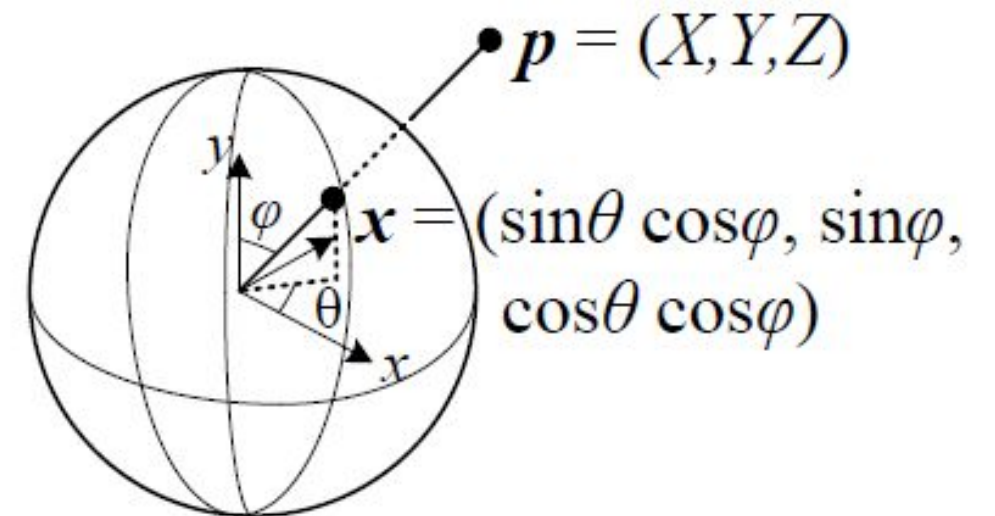
$$x' = s \cdot \tan^{-1} \left(\frac{x}{f} \right)$$

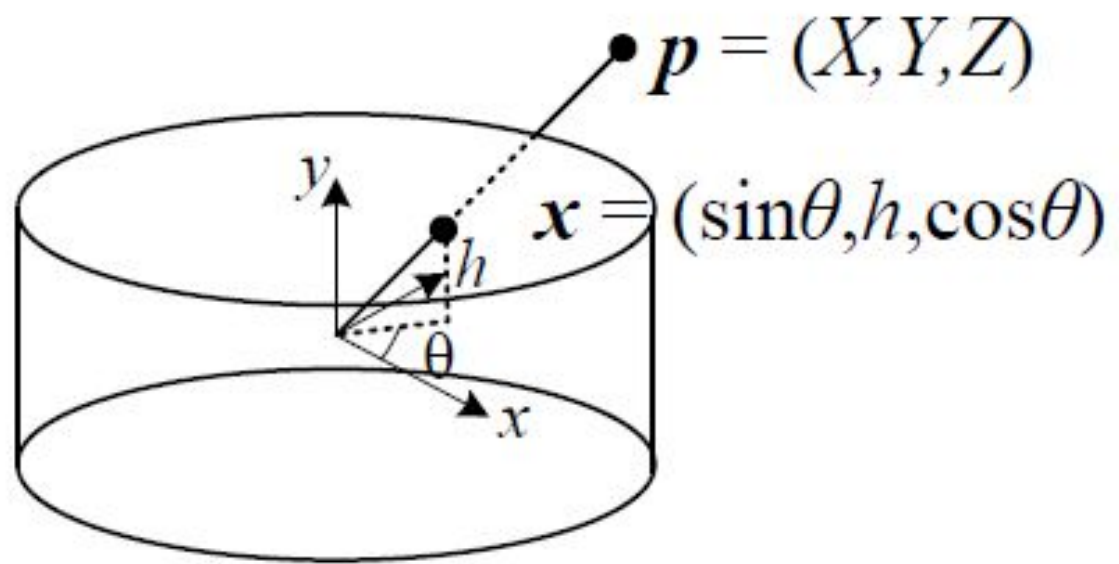
- Inverse mapping:

$$y' = s \cdot \tan^{-1} \left(\frac{y}{\sqrt{x^2 + f^2}} \right)$$

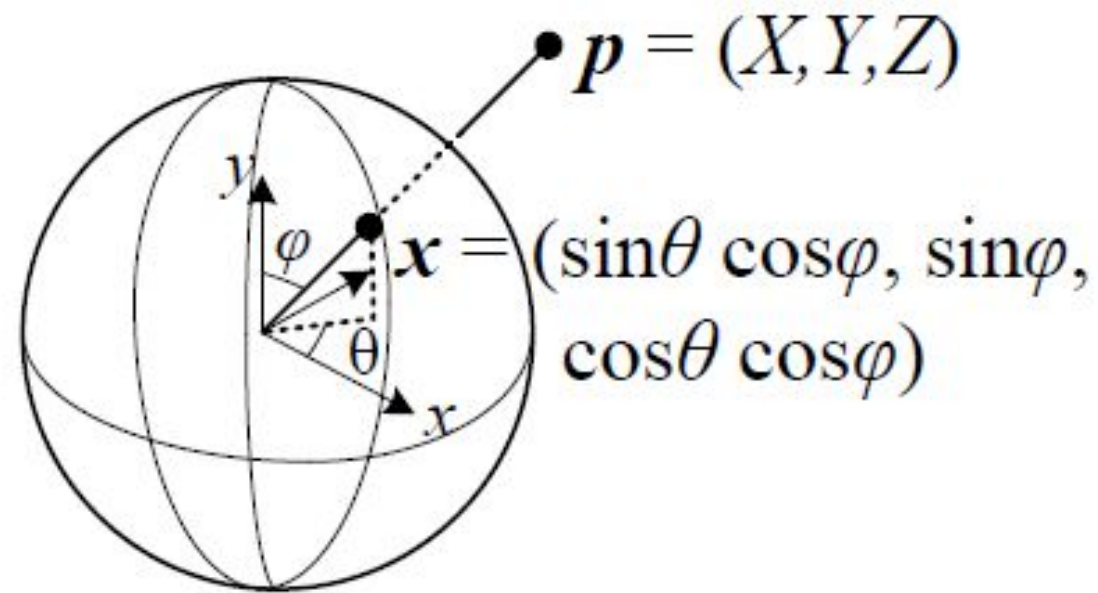
$$x = f \cdot \tan \left(\frac{x'}{s} \right)$$

$$y = f \cdot \tan \left(\frac{y'}{s} \right) \cdot \sec \left(\frac{x'}{s} \right)$$



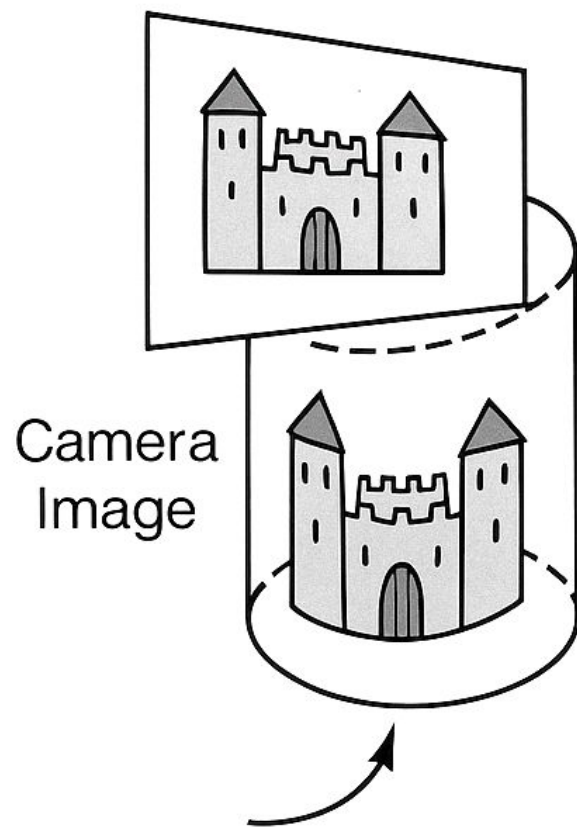


(a)



(b)

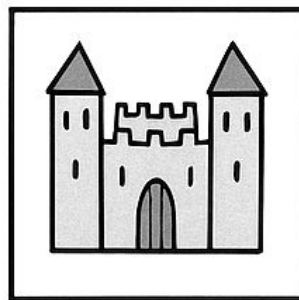
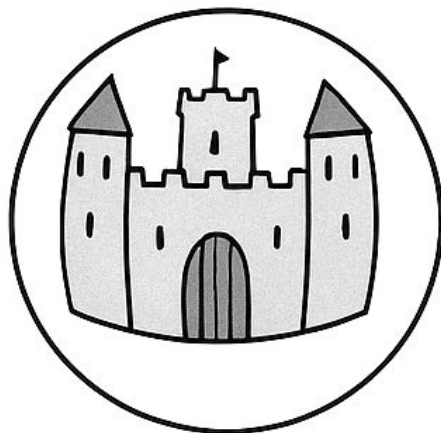
CYLINDRICAL



$$x' = s\theta = s \tan^{-1} \frac{x}{f}$$

$$y' = sh = \frac{y}{x^2} \sqrt{x^2 + f^2}$$

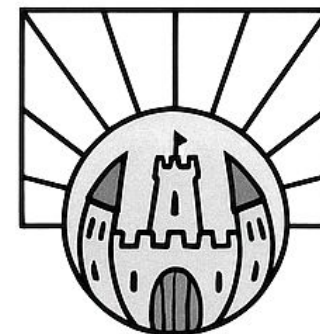
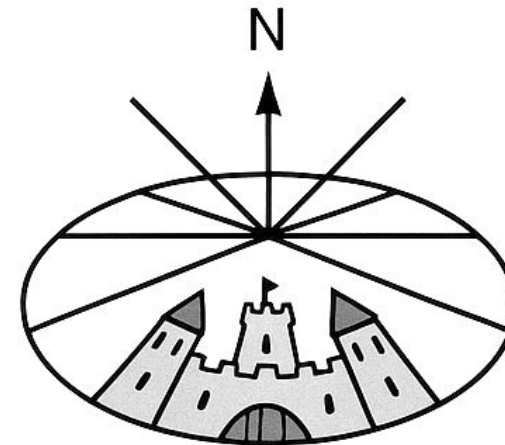
SPHERICAL



$$x' = s\theta = s \tan^{-1} \frac{x}{f}$$

$$y' = s\phi = s \tan^{-1} \frac{y}{\sqrt{x^2 + f^2}}$$

POLAR



$$x' = s\theta \cos \theta = s \frac{x}{r} \left(\frac{r}{z} \right)$$

$$y' = s\phi \sin \theta = s \frac{y}{r} \left(\frac{r}{z} \right)$$



(a)



(b)

- A cylindrical panorama (a) two cylindrically warped images related by a horizontal translation;
- (b) part of a cylindrical panorama composited from a sequence of images.

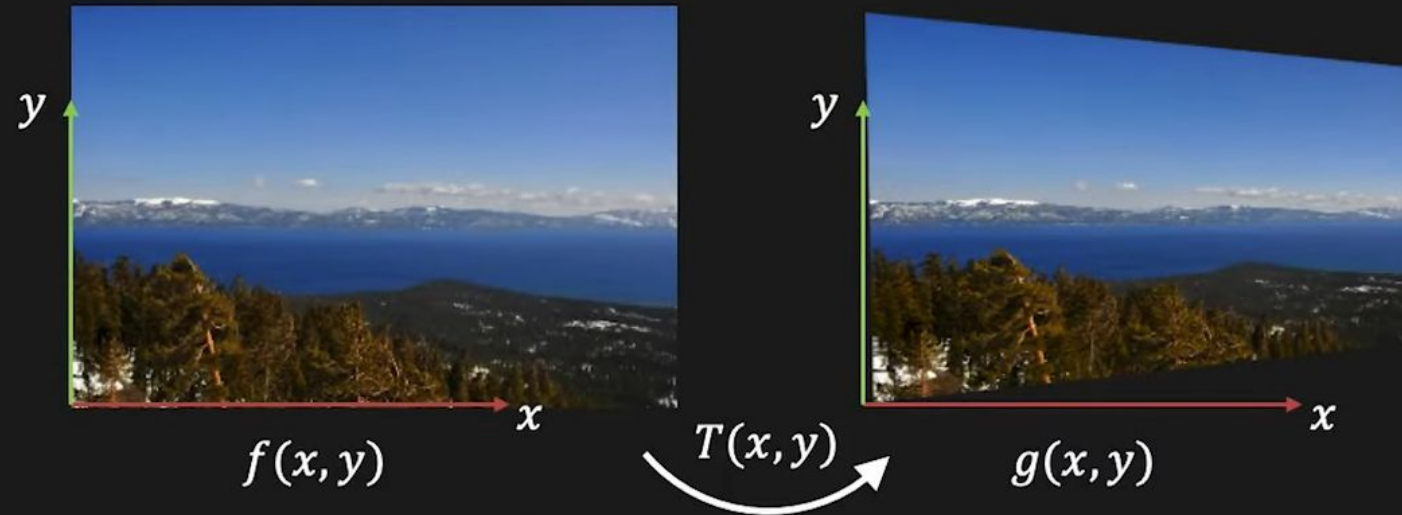
A spherical panorama constructed from 54 photographs



Warping

Given a transformation T and a image $f(x, y)$, compute the transformed image $g(x, y)$

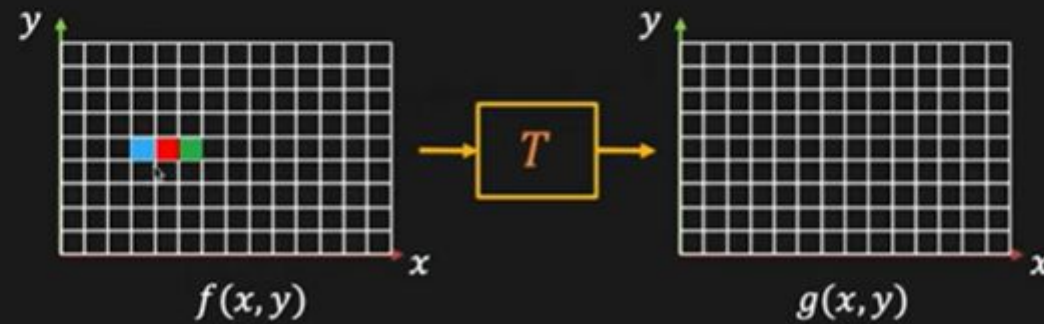
$$g(x, y) = f(T(x, y))$$



Forward warping

Send each pixel (x, y) in $f(x, y)$ to its corresponding location $T(x, y)$ in $g(x, y)$

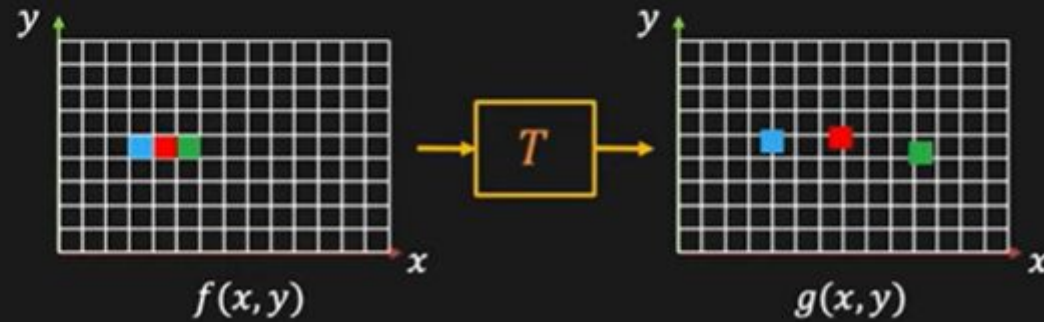
$$g(x, y) = f(T(x, y))$$



Forward warping

Send each pixel (x, y) in $f(x, y)$ to its corresponding location $T(x, y)$ in $g(x, y)$

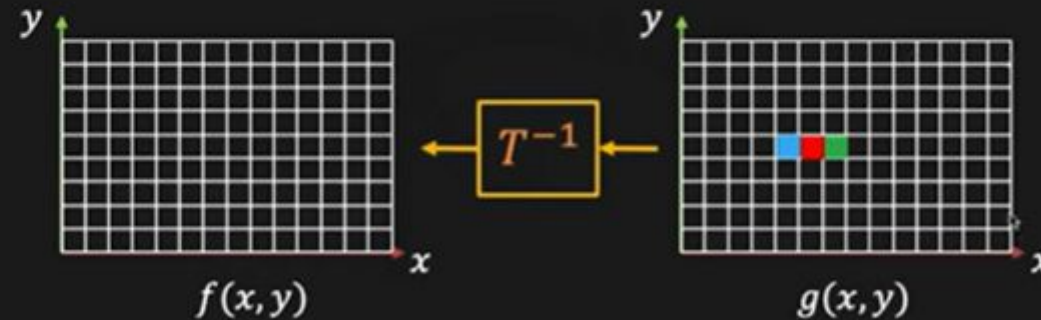
$$g(x, y) = f(T(x, y))$$



Backward warping

Get each pixel (x, y) in $g(x, y)$ from its corresponding location $T^{-1}(x, y)$ in $f(x, y)$

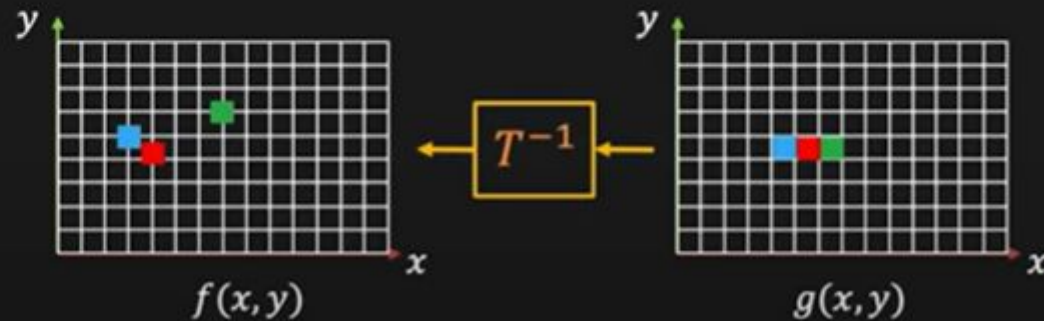
$$g(x, y) = f(T(x, y))$$

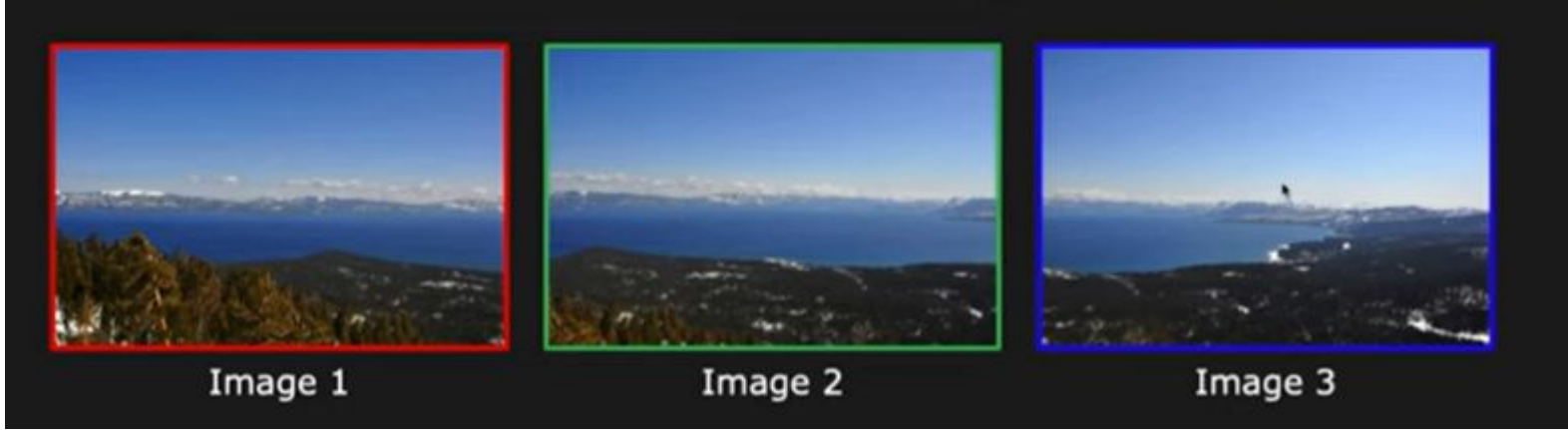


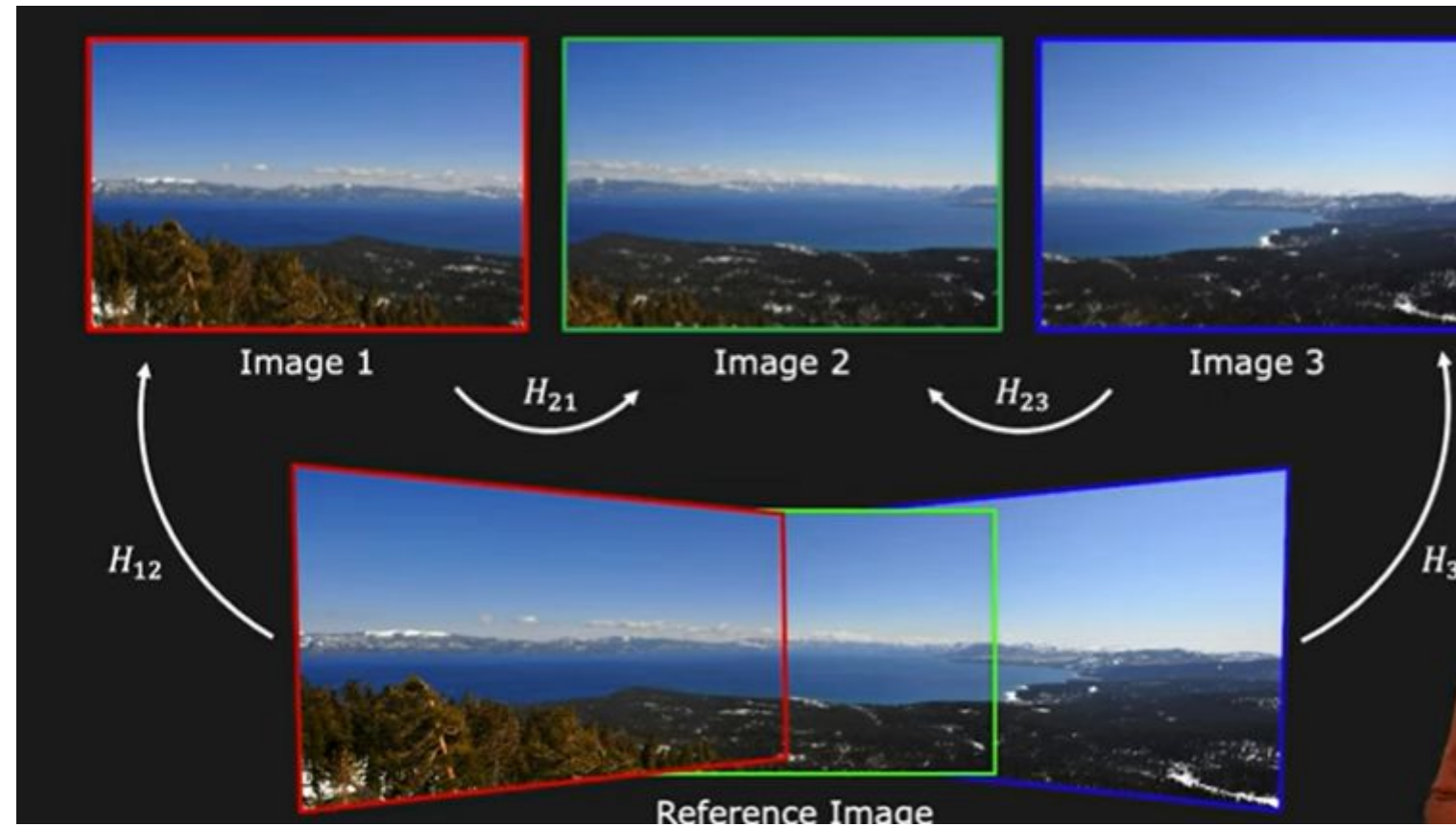
Backward warping

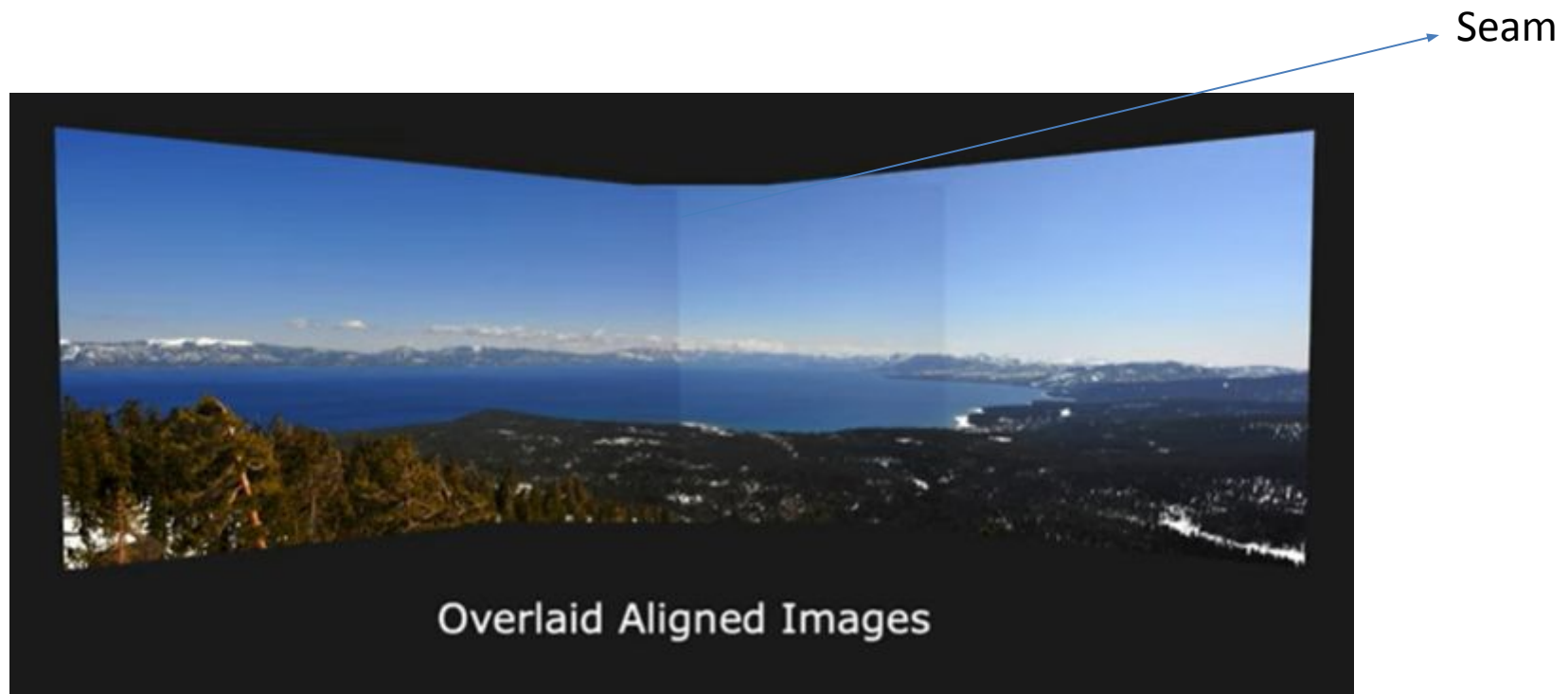
Get each pixel (x, y) in $g(x, y)$ from its corresponding location $T^{-1}(x, y)$ in $f(x, y)$

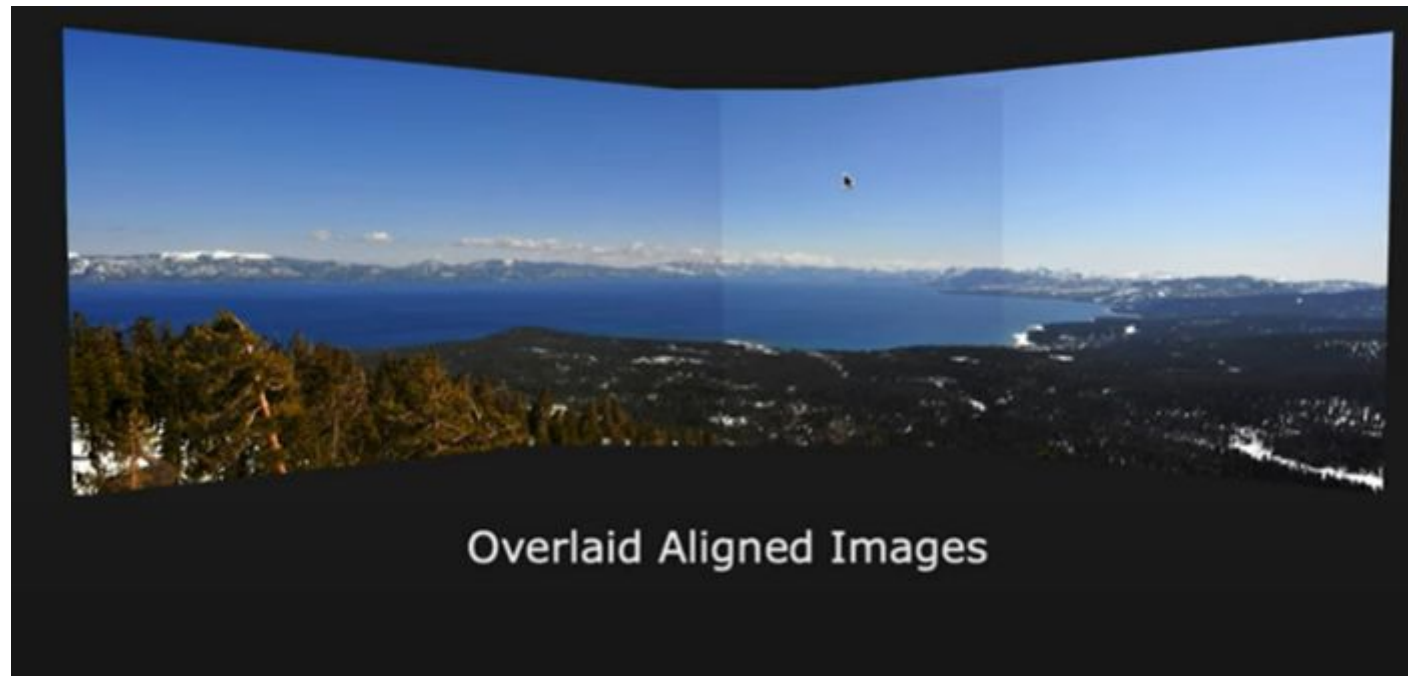
$$g(x, y) = f(T(x, y))$$













Averaged Images

Bundle adjustment

Parallax removal

Recognizing panoramas

GLOBAL ALIGNMENT

Global alignment

In the case of 360° *panoramas*, accumulated error may lead to the presence of a *gap* between the two ends of the panorama

This can be fixed by stretching the alignment of all the images using a process called *gap closing*

However, a better alternative is to simultaneously align all the images together using a **least squares framework** to correctly distribute any mis-registration errors.

The process of simultaneously adjusting pose parameters for a large collection of overlapping images is called *bundle adjustment*

In computer vision, bundle adjustment was first applied to the general **structure from motion** problem and then later specialized for **panoramic image stitching**

Why Global Alignment?

- When dealing with more than two images, **pairwise alignment** (aligning A to B, B to C, etc.) often results in **drift** or **accumulated errors** over time or space.
- To address this, **global alignment** aligns all images **relative to a common global reference**, optimizing all transformations together to minimize overall error.

Global Alignment

- **Global alignment** refers to the process of aligning **multiple images** (not just one pair) in a **globally consistent** way so that the entire panorama or image mosaic comes together smoothly — without visible seams, drift, or distortions.
- When you stitch a large set of overlapping images (e.g., for panoramas or satellite mosaics), aligning them **pair-by-pair** can cause **accumulated error** (also called **drift**).
- Global alignment fixes this by considering **all images and their relationships** together.

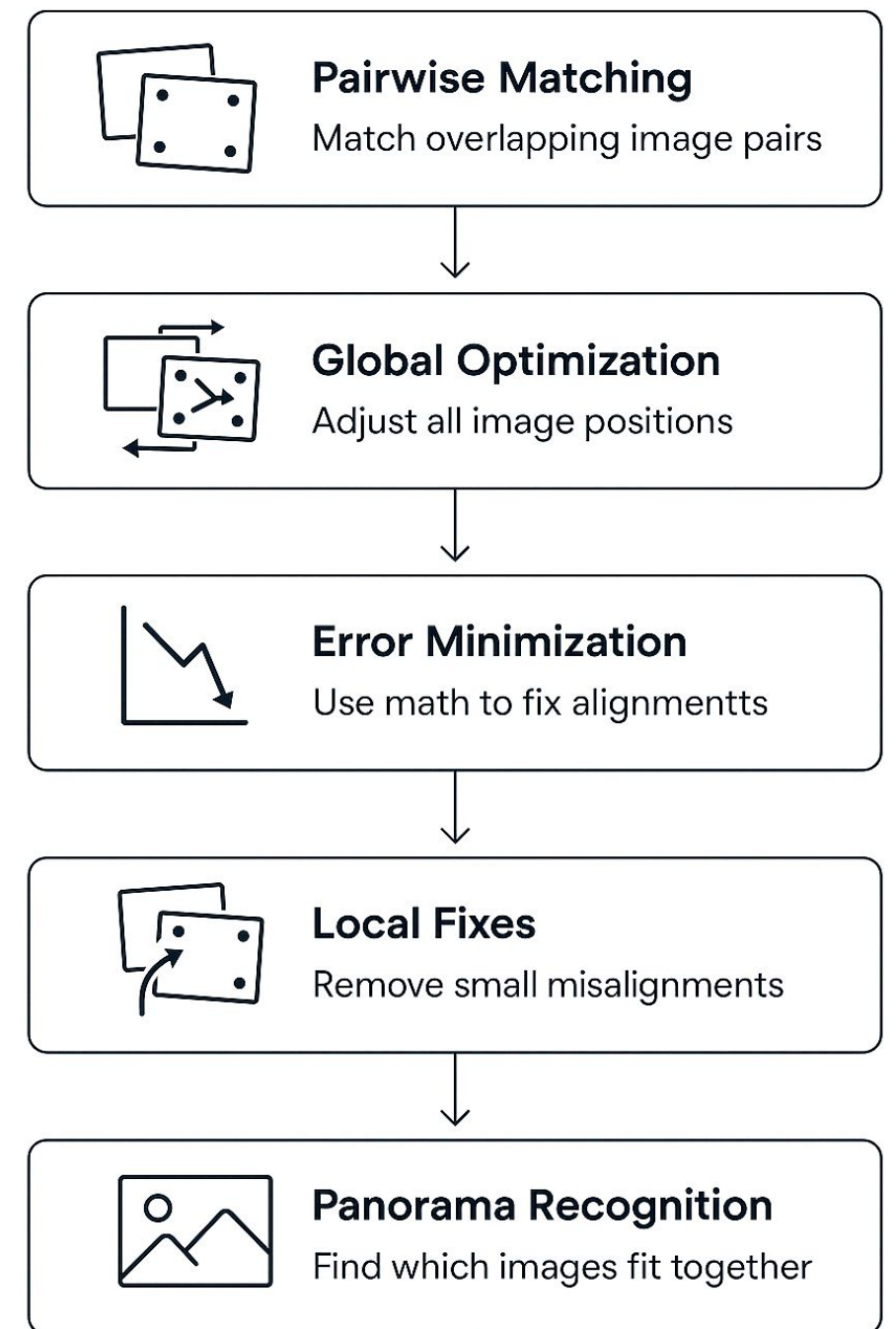
An **image mosaic** is a large image created by **stitching together multiple smaller overlapping images** so that they appear as a **single seamless panoramic image**.

Real-Time Example: Panoramic Image Stitching in Smartphones

- When you take a **panoramic photo** using your smartphone camera, you slowly sweep the phone across a scene. The camera captures **multiple overlapping frames** in real time.
- What's Happening Internally (Global Alignment)
 - **Frame-by-frame image capture** with overlapping regions.
 - **Feature detection and matching** between all pairs of frames (not just sequential ones).
 - **Homography estimation** between frames to find spatial relationships.
 - **Global optimization** to align all frames **relative to a central frame**, minimizing alignment drift.
 - **Warping and blending** of all images into a seamless panorama.

Steps in Global Alignment

- Define a Global Energy Function (Bundle adjustment)
- Local Adjustment (Parallax removal)
- Recognizing panoramas



Steps in Global Alignment

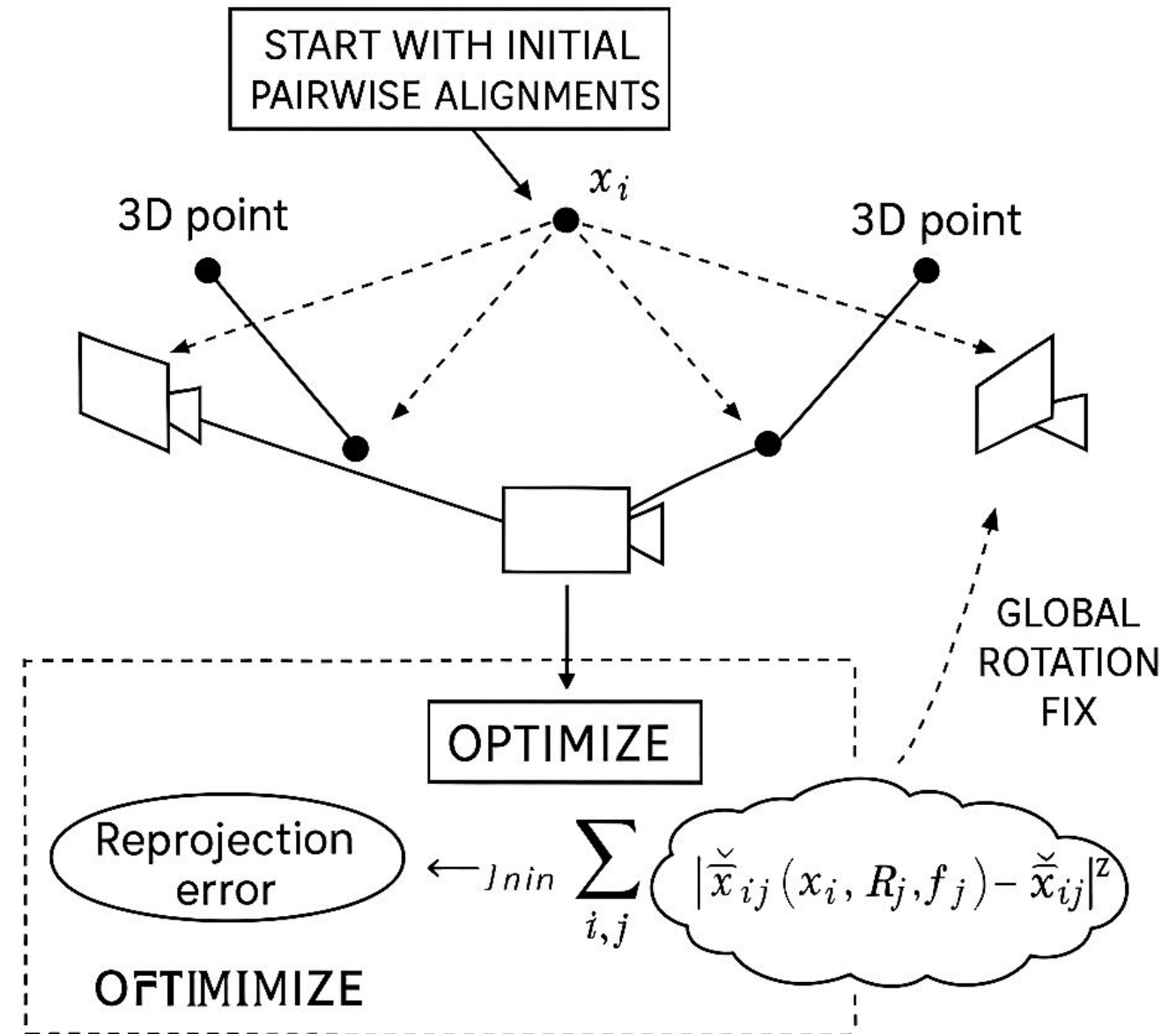
- **Step 1: Start with Pairwise Alignments**
 - First, we match **pairs of images** that overlap.
 - We find which parts in one image match parts in another (using feature points).
 - This gives us an **initial estimate** of how each image is positioned.
- **Step 2: Build a Global Alignment**
 - Now, instead of just fixing each pair, we want **all the images to fit together properly**.
 - We look at the **whole set of images together**, and adjust each one **slightly** to reduce overall mismatch.
- **Step 3: Minimize the Total Error**
 - We define an **error** — how far each matched point is from where it should be.
 - Then we use **math (optimization)** to move the images around and reduce this error across all pairs.

Steps in Global Alignment

- **Step 4: Fix Local Misalignments**
 - Even after global adjustment, some areas might still look blurry or doubled.
 - We apply **local adjustments**
 - Correct small shifts
 - Remove “ghost” images caused by parallax or movement
- **Step 5: Recognize Panoramas (if unordered)**
 - If we are given a **random set of images** (not in order), we:
 - Find out which ones overlap
 - Group them into one or more panoramas

Bundle adjustment

- **Goal:** Find a globally consistent set of alignment parameters that minimize the mis-registration between all pairs of images
- The process of simultaneously adjusting pose parameters for a large collection of overlapping images is called bundle adjustment in the photogrammetry community



Why is it called "Bundle" Adjustment?

- The term comes from the "**light rays**" (**bundles of rays**) connecting 3D points and their 2D projections in different camera views.
- **Bundle adjustment** "adjusts" the parameters so all these light rays best intersect their respective 3D points.

Real-Time Example: Visual SLAM in Autonomous Driving

- A **self-driving car** uses cameras to navigate urban environments. As it moves, it builds a **3D map** of its surroundings and estimates its own position in that map.
- This process is known as **Visual SLAM (Simultaneous Localization and Mapping)**.

Real-Time Example: Visual SLAM in Autonomous Driving

- How Bundle Adjustment Works
 - The car captures a video stream using onboard cameras (monocular or stereo).
 - Keypoints are detected and matched across consecutive frames.
 - Triangulated 3D points are generated from these matches.
 - **Initial camera poses and 3D point estimates** are noisy due to motion blur, lighting, or imperfect matches.
 - **Bundle Adjustment is run in real-time (or sliding windows) to:**
 - Refine the 3D structure of the scene.
 - Improve the estimated camera trajectory (car's position).
 - Minimize the reprojection error across many frames.

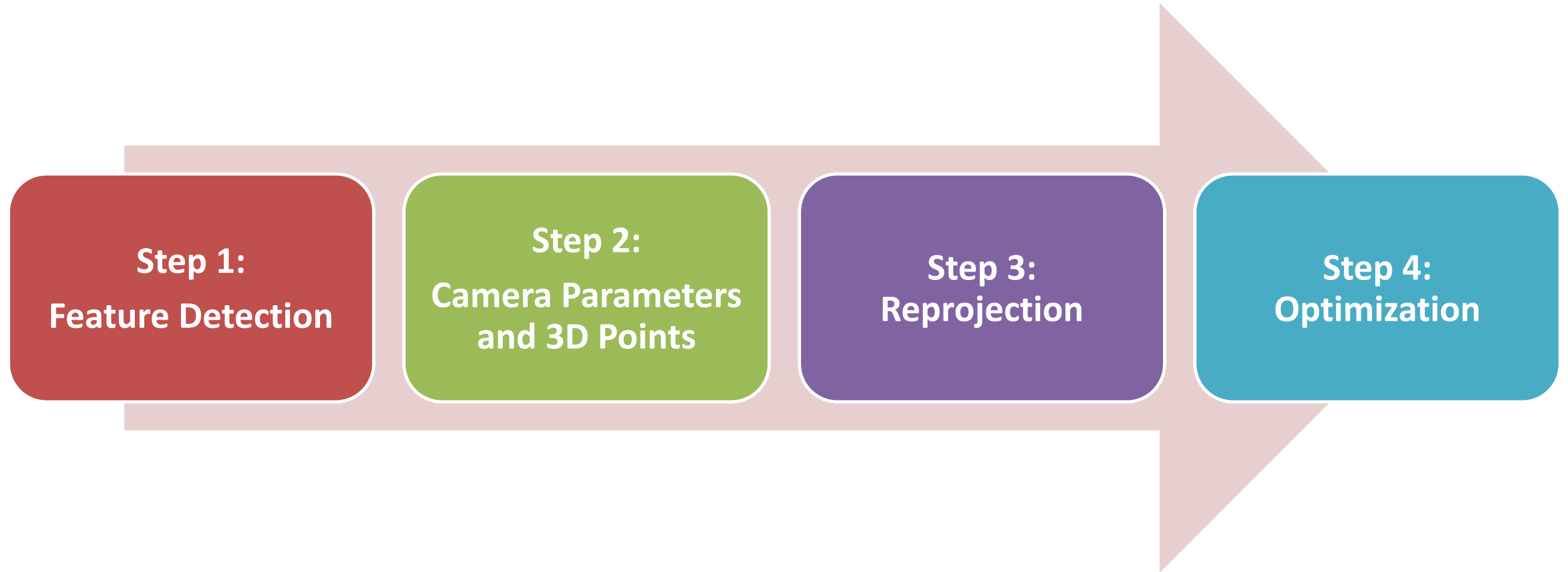
What is Bundle Adjustment?

- **Bundle Adjustment (BA)** is a mathematical process used to fine-tune the camera positions, rotations, and sometimes the 3D scene points to make sure all the images in a panorama (or a 3D reconstruction) align well — with **minimal error** across all views.
- It's like:
 - You start with a bunch of slightly misaligned photos of a scene.
 - You then **adjust every camera's position and orientation** together, so they all fit perfectly.

Why is it Needed?

- Imagine you're creating a **360° panorama**. If you only align each new image with the previous one (pairwise), small errors **accumulate** — leading to **gaps or overlaps** at the end.
- Bundle adjustment avoids this by:
 - Considering **all images at once**
 - Distributing the misalignment errors **evenly** and **optimally**

How Does It Work?



How Does Bundle Adjustment Work?

- **Step 1: Feature Detection**
 - You detect **matching points** (features) across multiple images.
 - Example: The top of a tower appears in 3 different images.
- **Step 2: Camera Parameters and 3D Points**
 - Each image has parameters:
 - **Rotation** R_j
 - **Focal length** f_j
 - Each feature has a guessed **3D position** x_i

How Does Bundle Adjustment Work?

- **Step 3: Reprojection**

- For every image and point, calculate where that 3D point **should appear** in 2D using the camera parameters.
- Compare this **predicted 2D point** with the **actual detected point** in the image.
- The **difference** is called the **reprojection error**.

- **Step 4: Optimization**

- Bundle adjustment **minimizes the sum of all these errors** across all images and all features.
- Uses **non-linear least squares optimization**.

Parallax removal

- **Parallax** is the apparent shift of objects when viewed from different positions.
- In the context of image stitching, it causes **misalignment**, **blurring**, or “**ghosting**” when stitching photos taken from slightly different positions (e.g., if the camera didn’t rotate exactly around its optical center).
- **What is Parallax?**
 - **Parallax** is the visual effect where the **position or direction of an object appears to change** when viewed from **different viewpoints**.
 - It’s caused by the change in **camera position** relative to the scene.

Parallax removal

- Once we have optimized the global orientations and focal lengths of our cameras, we may find that the images are still not perfectly aligned, i.e., the resulting stitched image looks **blurry or ghosted** in some places
- This can be caused by a variety of factors, including
 - Unmodeled **radial distortion**
 - **3D parallax** (failure to rotate the camera around its optical center)
 - **Small scene motions** (waving tree branches)
 - **Largescale scene motions** (people moving in and out of pictures)

Parallax removal: Solutions Based on Cause

1. Radial Distortion Correction

- Radial distortion can be estimated (potentially before the camera's first use) using
 - the *plumb line method* adjusts radial distortion parameters until slightly curved lines become straight,
 - Mosaic based approaches adjust them until mis-registration is reduced in image overlap areas

2. 3D Parallax Handling

- 3D parallax can be attacked by doing a full 3D bundle adjustment, i.e., replacing the projection, which models camera translations
 - The 3D positions of the matched features points and cameras can then be simultaneously recovered, although this can be significantly more expensive than parallax-free image registration.
 - Once the 3D structure has been recovered, the scene could (in theory) be projected to a single (central) viewpoint that contains no parallax.

Parallax removal: Solutions Based on Cause

3. Motion Compensation in the Scene

- **Large motion** (e.g., a person walking through): Just **ignore that region** in one of the images.
- **Small motion** (e.g., waving leaves): Use **optical flow** to estimate motion and apply **local warping**.

Local Alignment Method (Shum & Szeliski)

- This is a **post-bundle-adjustment refinement** to reduce local misalignments.
- **Process:**
 - **Back-project 3D features** from global bundle adjustment.
 - **Average their positions** to get target 3D point x_i .
 - **Project back** to each image to get **target image point** x_{ij} .
 - Compute **local motion**:
$$U_{ij} = x_{ij}^{\text{target}} - x_{ij}^{\text{original}}$$
 - Interpolate u_{ij} into a dense **motion field** $u_j(x)$ using bilinear kernels.
 - Warp the image to apply these local corrections.
- This helps reduce parallax and small local mismatches **without changing the original camera model**.

Alternative: Optical Flow De-Ghosting (Kang et al.)

- Compute dense optical flow between all images and a reference image.
- Use photo-consistency to **validate warped pixels**.
- Composite images based on reliable flow to remove ghosting.
- Works best for **HDR image stitching**, but less suited for general mosaics unless extended.

Why Parallax Happens

- **Causes:**
 - Camera didn't rotate around the exact optical center
 - Objects at different depths (e.g., foreground trees vs background buildings)
 - Slight camera shake or translation
 - Moving objects in the scene (like people or cars)

Problem: Misalignment After Global Optimization

- Even after perfect **global alignment** and **bundle adjustment**, you may see:
 - Ghosted objects
 - Blurry areas
 - Visible seams
- That's because **global models assume rigid scenes**, but **real scenes** are 3D and often contain motion.

How Parallax is Removed

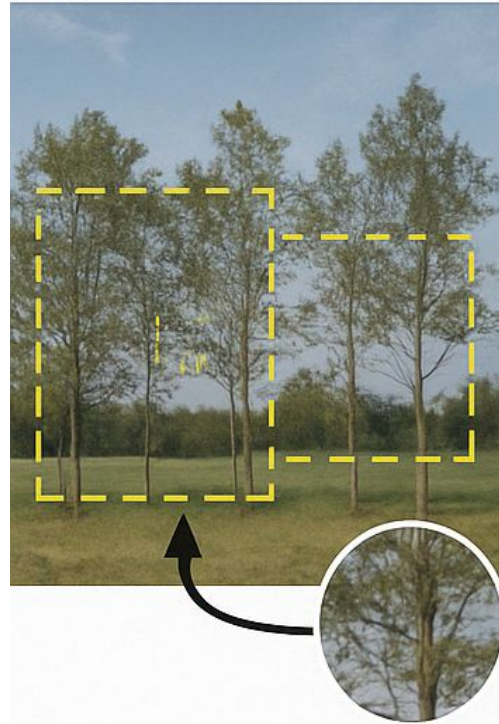
- **1. Estimate and Correct Radial Distortion**
 - Use methods like
 - **Plumb-line method** (straightens curved lines)
 - **Patch-based correction** (aligns overlapping areas)
 - Example: A lamp post looks bent across images — fix distortion so it appears straight.
- **2. Handle 3D Parallax via Bundle Adjustment + Dense Stereo**
 - Use **full 3D projection equations** (modeling camera translation)
 - Reconstruct **3D scene structure**
 - Project all points to a common **virtual viewpoint**

How Parallax is Removed

- **3. Use Multi-Perspective Plane Sweep (MPPS)**
 - Sweep a virtual camera through different depths
 - Composite best pixels from overlapping images based on depth match
- **4. Select Static Pixels Only**
 - For **moving objects**, just choose pixels from one image to avoid ghosts.
 - Like cutting one person out from a set of duplicates.
 - Example: In a busy street panorama, pick only one version of a walking person.
- **5. Apply Local Alignment (2D Optical Flow)**
 - Estimate **small motion vectors** in overlapping regions
 - Warp images **locally** to align better before blending
 - Uses **dense flow** or **feature patch grids**

Steps to Remove Parallax

Detect Problem
Areas



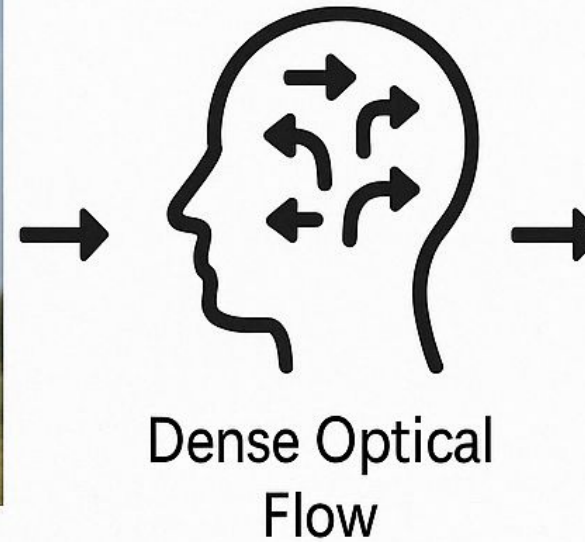
Ensure that
straight lines
remain straight

Correct
Radial Distortion




Ensure that
straight lines
remain straight

Choose Parallax
Correction Strategy



Adjust images
to align locally

Output




Ghost-free
panorama

Example: Parallax removal



(a)



(b)



(c)

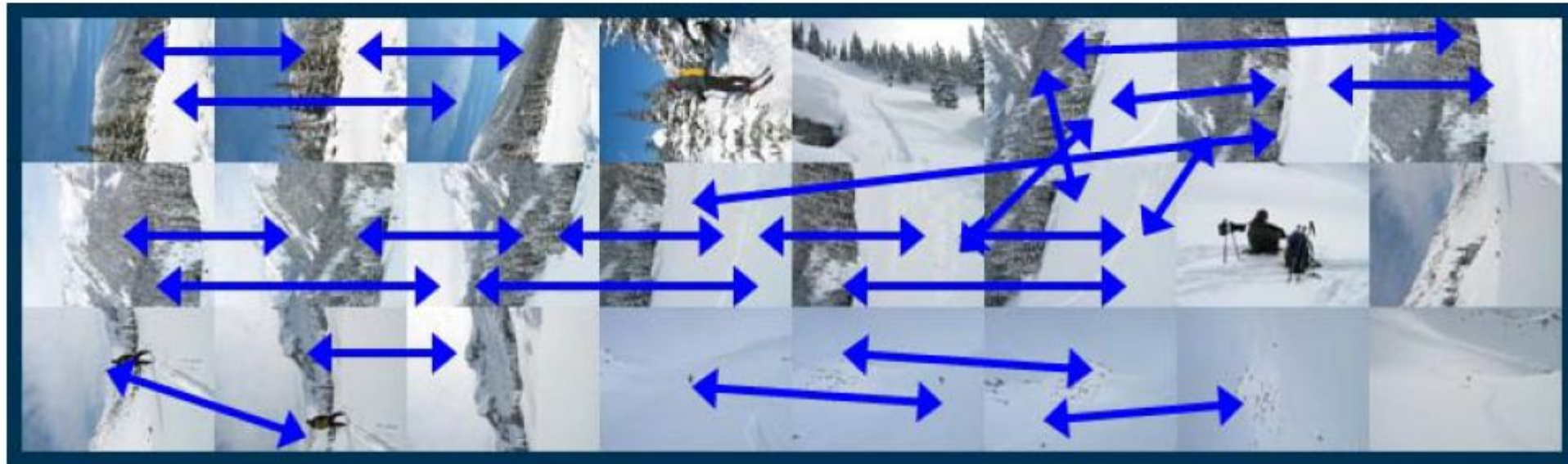
- Deghosting a mosaic with motion parallax
 - (a) composite with parallax;
 - (b) after a single deghosting step (patch size 32);
 - (c) after multiple steps (sizes 32, 16 and 8).

Recognizing panoramas

- It automatically determine **which images belong to the same panorama**—even when the user doesn't follow a clear, sequential pattern (like in handheld or 360° photography).
- **Problem:**
 - Users may:
 - Take images in **non-linear** order (e.g., top row, then bottom).
 - Jump **backward or forward** during capture.
 - Start new rows or restart capture at arbitrary locations.
 - Capture **360° panoramas**, where **start and end images overlap**.
 - So we need a system that can:
 - Automatically **group images into panoramas**, and
 - **Determine the correct stitching order** even when unordered.

Recognizing panoramas

- The final piece needed to perform fully automated image stitching is a technique to recognize which images actually go together, which is called recognizing panoramas



How Does It Work? Brown and Lowe's Approach (2007)

- **Step 1: Feature Extraction**
 - Use **SIFT** (Scale-Invariant Feature Transform) to extract **keypoints and descriptors** from each image.
 - These descriptors describe image corners, edges, blobs, etc., in a way that is **invariant to scale and rotation**.
- **Step 2: Match Features Between All Image Pairs**
 - For every image pair, find **corresponding features** using nearest-neighbor search in descriptor space.
 - Apply **RANSAC** to filter outliers and compute a **valid geometric model** (like a homography or rotation).

How Does It Work?

- **Step 3: Build an Overlap Graph**

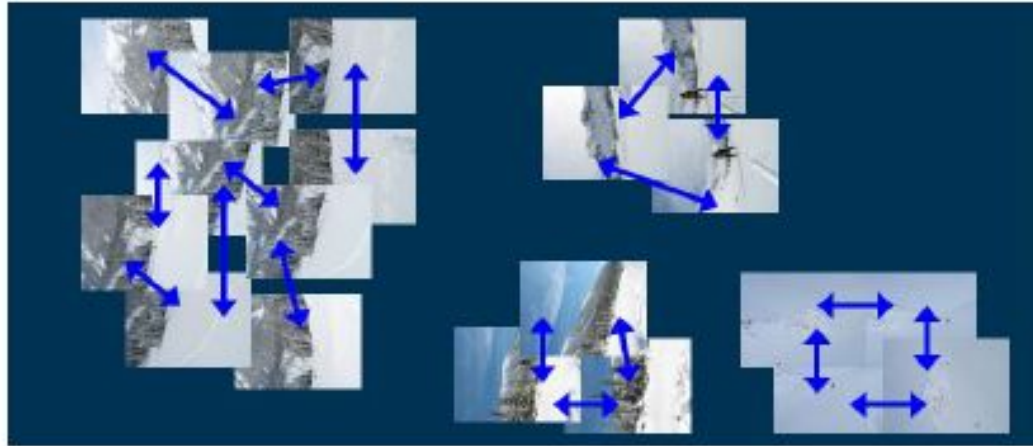
- Construct a graph where:
 - **Nodes** = Images
 - **Edges** = Valid overlaps between images (based on matched features)
- Use this to find **connected components**, where each component likely represents a **distinct panorama**.
- Example: If image A overlaps with B, and B with C, but not with D — then {A, B, C} is one panorama.

- **Step 4: Validate Panorama Groups**

- Check if the matching images really show the **same scene**:
 - Do a **pixel-based comparison** for verification.
 - Apply heuristics to detect **false positives** caused by repeated structures (e.g., windows, tiles).



(a)



(b)



(c)

- Recognizing panoramas
 - (a) input images with pairwise matches;
 - (b) images grouped into connected components (panoramas);
 - (c) individual panoramas registered and blended into stitched composites.

COMPOSITING

Compositing

- Choosing a Compositing Surface
 - View Selection
 - Coordinate Transformations
 - Sampling Issues
- Pixel Selection and Weighting
 - Feathering and Center-Weighting
 - Optimal Seam Selection
- Blending
 - Laplacian Pyramid Blending
 - Gradient Domain Blending
- Exposure Compensation
 - High Dynamic Range Imaging

What is Compositing?

- **Compositing** is the process of **merging aligned images** into a single seamless mosaic or panorama. Once the images are registered (aligned in space), compositing decides:
 - **Where** to place the final image on a surface (flat, cylindrical, spherical)
 - **Which pixels** to take from each image in overlapping regions
 - **How** to blend those pixels to make transitions **smooth and natural**

Steps in Compositing

- **1. Choose a Compositing Surface**
 - This defines the geometry of the final panorama.
 - **Flat plane:** Used for small panoramas.
 - **Cylindrical:** Good for wide-angle horizontal panoramas.
 - **Spherical:** Required for 360° full-view environments.
 - Depends on camera rotation type and the desired output experience (e.g., immersive VR).
- **2. Select the Reference View**
 - One image (often the **central** one) is chosen as the **anchor view**.
 - All other images are warped relative to this view to preserve perspective and avoid distortion.

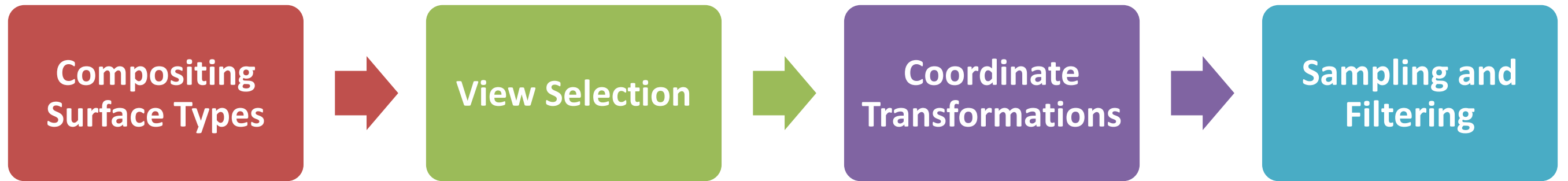
Steps in Compositing

- **3. Pixel & Seam Selection**
 - In overlapping regions, the system must choose:
 - **Which pixels to keep** from which image.
 - Where to **draw seams** so that transitions are least visible.
 - Techniques include:
 - **Graph cuts**: Finds seams through areas with least visible difference.
 - **Dynamic programming**: Optimizes pixel selection based on smoothness and consistency.
- **4. Blending**
 - This removes seams and ensures **color and tone consistency**.
 - Techniques:
 - **Feathering**: Linear blending of pixel values.
 - **Multi-band blending** (Burt and Adelson): Blends images at multiple frequency levels to handle illumination differences.
 - **Gradient domain blending**: Preserves edges and structures during blending.

Steps in Compositing

- **5. Exposure Compensation**
 - Handles inconsistencies in brightness or color between images due to:
 - Auto-exposure variation
 - Lighting changes during capture
 - Adjustments:
 - Histogram matching
 - Gain and bias tuning
 - Local contrast normalization

1. Choosing a Compositing Surface



1. Choosing a Compositing Surface

1. Compositing Surface Types

- The final mosaic needs to be projected onto a **surface** to maintain geometric consistency. The options include:
- **Flat (Planar) Surface:**
 - Best for **small fields of view (FOV)**.
 - Uses a **perspective projection**: straight lines remain straight.
 - One input image is selected as the **reference**, and others are warped to its coordinate system.
 - Distortion becomes significant if FOV exceeds $\sim 90^\circ$.
- **Cylindrical or Spherical Surfaces:**
 - Better suited for **large or 360° panoramas**.
 - Prevents extreme pixel stretching near image borders.
 - Common in **VR and full-scene visualization**.
- **Cube Maps and Other Polyhedra:**
 - Often used in **environment mapping** (as in graphics/gaming).
 - Provide a structured way to cover the full viewing sphere using cube faces.
- **Cartographic Projections:**
 - Alternative mappings developed in geography to represent the globe.
- **Tradeoff:** There's a balance between **local geometric accuracy** (e.g., keeping lines straight) and **even spatial sampling**.

1. Choosing a Compositing Surface

2. View Selection

- Once the surface is chosen, the system needs to **choose a reference view or direction** for compositing:
- For **flat panoramas**, typically choose:
 - The most **geometrically central image**, e.g., one whose **z-axis** aligns best with the average z-axes of all images.
 - Or the **middle image** in a sequence.
- For **rotational or full 360° panoramas**:
 - First image can be chosen if it contains the most important object.
 - Automatic methods or **user interaction** (e.g., panning) can help define the final view.

Choosing a Compositing Surface

3. Coordinate Transformations

- Transforming each input image to the chosen surface involves:
- **Flat Surfaces:**
 - Apply a **homography** (as in Eq. 2.19).
 - Easily implemented with hardware-accelerated texture mapping.
- **Cylindrical/Spherical Surfaces:**
 - Each pixel in the panorama maps to a **3D ray**.
 - Project back to input images via **camera models** (including distortion correction).
 - Can optimize with **lookup tables** or **grid interpolation** for speed.
- **Polyhedral Surfaces (e.g., Cube Maps):**
 - Requires 3D transformations and care to handle texture seams and valid pixel regions.

1. Choosing a Compositing Surface

4. Sampling and Filtering

- Even with correct pixel mappings, **aliasing and sampling artifacts** can occur:
- If the panorama has lower resolution, **pre-filtering** is necessary.
- Techniques include:
 - **MIP Mapping** (multi-resolution texture pyramids).
 - **Elliptically Weighted Averaging** (for anisotropic sampling).
 - **Cubic Interpolation with Adaptive Pre-filtering** for high quality.
- In some cases, **super-resolution** techniques may be applied to synthesize higher-resolution output than the original images.

2 Pixel Selection and Weighting

- It delves into how to determine *which pixels* from overlapping images contribute to each part of a panorama and *how to blend* them to create seamless results—especially in the presence of real-world artifacts like exposure variation, ghosting, or misalignment.



2 Pixel Selection and Weighting

- **The Problem of Real-World Stitching**

- Even when images are geometrically aligned (registered), blending them is non-trivial due to:
 - **Exposure mismatches** → visible seams.
 - **Slight misalignments** → blur.
 - **Moving objects** → ghosting.

2 Pixel Selection and Weighting

Basic Pixel Blending Methods

- **Simple Averaging:**

- Use a weighted average at each pixel:

$$C(x) = \frac{\sum_k w_k(x) \cdot \tilde{I}_k(x)}{\sum_k w_k(x)}$$

- $\tilde{I}_k(x)$: re-sampled image.
- $w_k(x)$: binary weight (1 = valid pixel, 0 = invalid).

- Implementable in graphics hardware (accumulation buffer).
- **Issue:** Not robust—sensitive to exposure and motion differences.

2 Pixel Selection and Weighting

- **Feathering & Distance-Based Weighting**

- To reduce artifacts:
- **Feathering**: Smooth transition between images using **distance maps**.
 - Compute Euclidean distance from each valid pixel to its nearest invalid pixel:

$$w_k(x) = \min_y \left\{ \|y\| \mid \tilde{I}_k(x + y) \text{ is invalid} \right\}$$

- Better blending but may still blur or ghost.
- **Power Weighting (p-norm approximation)**:
 - Raise weights to power p to emphasize dominant pixels.
 - As $p \rightarrow \infty$, results in **hard pixel selection** (like a Voronoi diagram):

$$C(x) = \tilde{I}_{l(x)}(x), \quad l(x) = \arg \max_k w_k(x)$$

2 Pixel Selection and Weighting

Seam Placement Strategies

- **Voronoi Diagrams:**
 - Assign pixels to closest image center (via max-weight rule).
 - Fast but **can create harsh seams** with exposure differences.
- **Optimal Seam Placement:**
 - Prefer placing seams **where images agree** (low intensity or gradient difference).
 - Use **Dynamic Programming (DP)** for seam finding between **two images**.
 - For **multiple images**, DP doesn't generalize easily.
- **Graph-Based Seam Optimization (Uyttendaele et al.):**
 - Identify **Regions of Difference (RODs)** (areas of misalignment or object motion).
 - Create a **graph**:
 - Nodes = RODs
 - Edges = overlaps between RODs
 - Use **vertex cover algorithm** to remove minimum necessary RODs.
 - Prefer to remove RODs near image edges (via weighted vertex cover).
 - Once RODs are selected for removal, use **feathering** to blend remaining pixels.

2 Pixel Selection and Weighting

MRF-Based Pixel Labeling (Agarwala et al.)

- Use an **MRF energy minimization** framework:
- **Objective 1: Data Cost (Pixel Suitability)**
 - Penalizes choosing bad pixels.
 - Can include user inputs or automated strategies (e.g., likelihood-based).
- **Objective 2: Seam Cost (Transition Smoothness)**
 - Penalizes label changes (i.e., seams) between neighbors.
 - Can be based on:
 - Color differences.
 - Gradient differences.
 - Edge alignments.
- **Total Energy (to minimize):**
$$E = C_D + C_S$$
 - Solved using **graph cuts** (e.g., α -expansion method).
 - Results in **globally optimized labelings**.
 - Graph cuts are **accurate but computationally expensive**.

$$C_D = \sum_x D_{l(x)}(x)$$

$$C_S = \sum_{(x,y) \in N} S_{l(x),l(y)}(x,y)$$

2 Pixel Selection and Weighting

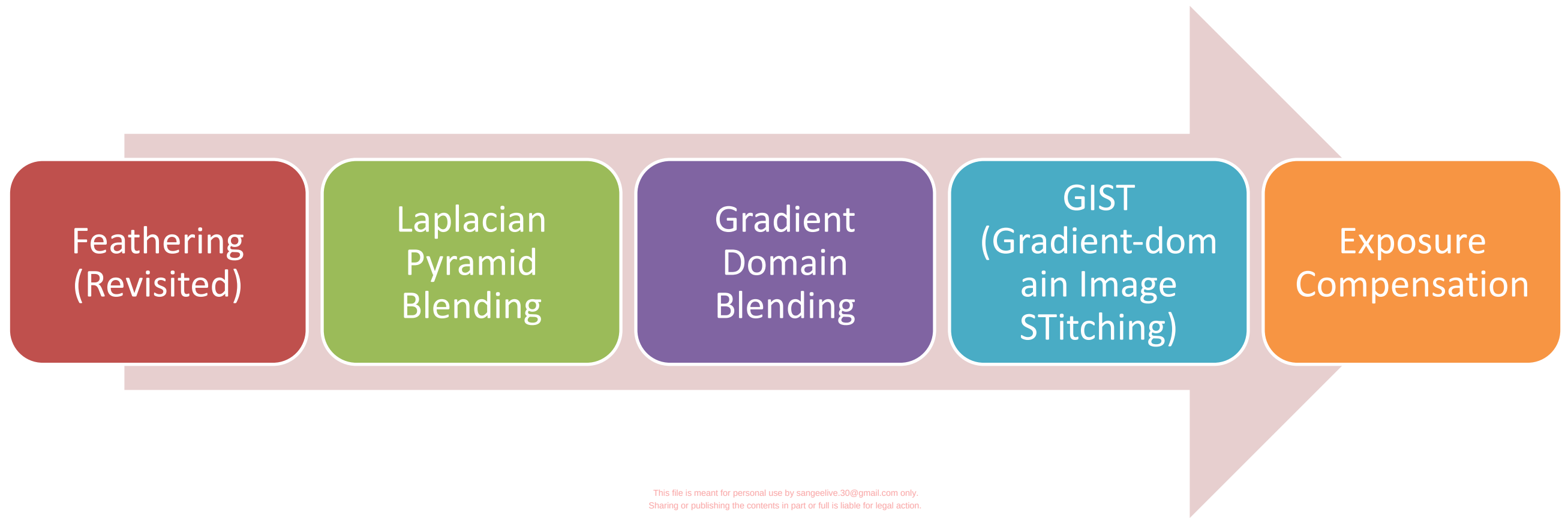
- **Alternative Seam Techniques**

- Watershed Algorithms
 - Use edge strength to direct seam placement (Soille).
- Median Filtering
 - For eliminating transient (moving) objects.
- Center-Weighting & Likelihood Methods
 - Retain or suppress rare objects.



3. Blending

This final part of the image stitching pipeline focuses on **blending techniques**, which aim to produce a **visually seamless panorama** despite challenges such as exposure variations, slight misalignments, and moving objects.



3. Blending

- **Feathering (Revisited)**
 - Spatially varying weights are used to blend overlapping pixels.
 - Works well for minor exposure differences.
 - **Problem:** May blur sharp transitions or fail in the presence of large intensity differences.
 - Raising weights to a power helps emphasize dominant contributions (p-norm effect).

3. Blending

- **Laplacian Pyramid Blending**
 - A **multi-band, frequency-adaptive approach** to blending.
 - **Steps**
 - **Build Laplacian Pyramid** for each warped input image
 - Smooth image using binomial kernel.
 - Downsample.
 - Subtract to get band-pass (detail) layers.
 - **Build Gaussian Pyramid** of the mask (valid pixel regions) for each image.
 - **Per-level blending**: Feathered blending at each level using Gaussian mask.
 - **Reconstruction**: Upsample and sum all pyramid levels to get the final image.
 - **Advantages**
 - Smoothly blends low and high frequencies.
 - Reduces exposure seam visibility while maintaining detail.

3. Blending

- **Gradient Domain Blending**

- Rather than directly blending **pixel values**, this technique blends **image gradients**, and then reconstructs the image.
- **Key Concepts**
 - First proposed for **seamless cloning**
 - Solve **Poisson equation** using either:
 - **Dirichlet boundary** (force match at seam),
 - Or **Neumann boundary** (no assumption about final value).

- **Optimization Formulation**

- Minimize difference between gradient of final image and selected source image:

$$\min_{C(x)} \left\| \nabla C(x) - \nabla \tilde{I}_{l(x)}(x) \right\|^2$$

- Solved via sparse linear systems.
- Hierarchical solvers (e.g., multigrid or preconditioned conjugate gradient) improve efficiency.

- **Benefits**

- Eliminates seams.
- Can blend illumination, shadows, and reflections naturally.

3. Blending

- **GIST (Gradient-domain Image STitching) (Levin et al. [116])**
 - A refined gradient domain method.
 - **Variants studied:** L2 vs. L1 norm, feathering of gradients, and iterative reconstruction.
 - Preferred method: **L1-optimized blending of feathered gradients (GIST1-l1).**
 - Faster implementation: **iterative median filtering** in multigrid framework.
 - **Strengths:**
 - Outperforms both Laplacian and feathering for complex real-world blends.
 - Avoids double edges or ghosting better than naive gradient methods.

3. Blending

- **Exposure Compensation**
 - **Iterative Local Compensation**
 - Block-based fitting between input image and a feathered composite.
 - Smooth the mapping via neighborhood averaging.
 - Interpolate (spline) between blocks for pixel-level adjustment.
 - Re-blend and repeat for convergence.
 - **Use Case**
 - Effective for local exposure variations (e.g., lens vignetting).
- **HDR-Based Blending (High Dynamic Range Imaging)**
 - When exposure differences are large:
 - **Radiance Map Estimation:**
 - Estimate a true scene **radiance map** $R(x)$ from multiple exposures $I_k(x)$.
 - Use inverse camera response functions:

$$I_k(x) = f(c_k R(x); p)$$



(a)



(b)



(c)



(d)



(e)



(f)



(g)



(h)

- Final composites computed by a variety of algorithms:
 - (a) average,
 - (b) median,
 - (c) feathered average,
 - (d) p-norm $p = 10$,
 - (e) Voronoi,
 - (f) weighted ROD vertex cover with feathering,
 - (g) graph cut seams with Poisson blending
 - (h) with pyramid blending.

Tutorial

- T9: Case Study
- T10: Case Study

QUIZ